SWE5006 PRACTICE PROJECT · TEAM 8

BLOCBUY

ONE FOR ALL • ALL FOR ONE

A community group-buying platform with pay-first teams

Two prices • pay first to team up • refunded if the team doesn't fill

1 Introduction · 2 Analysis, design, patterns · 3 Testing · 4 CI/CD demo · 5 System demo

Lim Jiechao · Liu Jin · Panpakorn Siripanich · Wang Ziang · Yu Qianshuo

Graduate Certificate in Designing Modern Software Systems · 26 October 2026

Group buying fails when buyers cannot commit together

> **Small merchants can discount only against committed volume, and no single buyer can reach it: demand exists, but nobody commits first.**

Today

- Merchants discount only when volume is predictable
- One buyer can never reach a bulk price
- Advertising is costly and not tied to real purchases

BlocBuy: pay-first teams

- Two prices: individual and team
- Pay upfront to start a team, then share the invite link
- Friends join by paying upfront too
- Fills in time: team price. Expires: everyone refunded automatically

Stripe test mode only (proposal §6.1)

JC Lim

Scope: nine use cases, six quality attributes with targets

Report [FILL: § (#463)] · scope, quality attributes · RT 1.4.1–1.4.3

IN SCOPE · PROPOSAL §6.1

UC1 Browse and view listings
UC2 Register / authenticate
UC3 Initiate a team (pay first)
UC4 Join a team
UC5 Team lifecycle resolution
UC6 Refund on team expiry
UC7 Share attribution
UC8 View my orders and teams
UC9 Admin listing management

+ Receive notifications (Sprint 4, SWE5001; off on dev)

OUT OF SCOPE · PROPOSAL §6.2

Merchant self-service · analytics · recommendations · automatic team matching · email, SMS, push · native apps · live payments

QUALITY ATTRIBUTES AND TARGETS · PROPOSAL §6.3

Attribute	Target	Shown in
Correctness under concurrency	team capacity never exceeded; no payment double-charged or double-refunded	**§5 demo · §3**
Performance	interactive operations ≤ 2 s at 100 concurrent users; expiry resolved ≤ 1 min after the window	**§5 k6 run · §3**
Security	Google sign-in; role-based admin; signed Stripe webhooks; no credentials in code	**§5 demo · §4**
Maintainability, extensibility	new pricing, payment provider or team consequence without editing domain logic	§2 patterns · §4
Code quality	SonarCloud gate on every PR; unit tests on the complex classes	§3 · §4
Auditability	every payment, refund and transition traceable; every artefact traceable to a commit	§4

JC Lim

Promised, delivered, changed, dropped: status on 21 October

Report [FILL: § (#463)] · status, promised vs delivered · RT 1.4, 2.2

Promised (proposal)	Status on 21 Oct	Record
UC1–UC9	DELIVERED all nine closed as completed	#24–#32
UC9 admin API only, no GUI	CHANGED admin console built in Sprint 3	F-ADR 0015 · #305–#307
Adapter (UC6), Observer (UC7)	CHANGED gateway ports; Observer on UC5 → UC6	slides 10, 18 · *no ADR*
Pay-first START, JOIN, refund on dev	OPEN Checkout returns; webhook never confirms	#517 · #223
State for the team lifecycle	CHANGED transition table; State on listings	design doc · *no ADR (item 7)*
Google sign-in with JWT	CHANGED Sa-Token session, HttpOnly cookie	D6/D7, F-ADR 0002 · *no backend ADR*
Spring Data JPA	CHANGED MyBatis behind repository ports	D1/D9 · *no ADR (item 7)*
Dev on ECS Fargate and RDS	CHANGED one EC2 host, Compose, DB on host	ADR 0008
Secrets Manager, no static keys	CHANGED SSM and a root-only env file; 2 IAM users, 1 key	operator guide §7 · *no ADR*
Blocking OWASP and IaC scans	CHANGED Trivy deps and config, not required; 3 per-job roles	security.yml, ruleset · *no ADR*
Gated prod, ECS circuit breaker	DROPPED for SWE5006; built for SWE5001	#226 · ADR 0008
E2E, burndown, saved plans	PARTIAL stubs; daily data, no chart; manual	F-ADR 0011 · #229

Outstanding #517 and #223 (go/no-go 23 Oct) · SSH 22 open on the dev host · CodeQL off · CloudFront → ALB over HTTP until 16 Nov (#227) · dev volume unencrypted

Milestones achieved and effort per member

Report [FILL: § (#463)] · milestones, effort · RT 2.3

MILESTONES ACHIEVED

Date	Milestone
27 Aug	Proposal submitted
5 Sep	Backend pipeline live on dev
14 Sep	Sprint 1 tag
20 Sep	Stripe Sandbox acceptance (local)
22 Sep	Web app live on dev
27 Sep	Sprint 2 tag
30 Sep	Admin console merged (#305–#307)
11 Oct	Sprint 3 tag
23 Oct	Sprint 4 tag [FILL: date]
—	Paid start, join, refund on dev: not yet

EFFORT PER MEMBER · PLANNED HOURS [VERIFY: S4 split, PR #538]

Member	S1	S2	S3	S4	Planned	Actual
Liu Jin	—	30	30	51	**111**	[CONFIRM: agreed]
Yu Qianshuo	—	30	30	48	**108**	[CONFIRM: agreed]
Wang Ziang	—	30	30	46	**106**	[CONFIRM: agreed]
Panpakorn	—	30	30	41.5	**101.5**	[CONFIRM: agreed]
Lim Jiechao	—	30	30	38	**98**	[CONFIRM: agreed]

Sprint 1 hours were not recorded. Team done: S2 134 of 150 h, S3 117 of 150 h. Budget in the proposal: 10 man-days each (§7).

Planned, not actual: no member logged actual hours, and the progress reports carry none. [CONFIRM: actual effort agreed by all five]

Ownership: one use case and one design problem each

Report [FILL: § (#463)] · ownership · RT 2.1, 3.4, 3.5

Member	Use case	Design problem → GoF pattern	Slide	Change from proposal §6.1
Liu Jin	UC9 admin listings and team terms	Listing lifecycle → **State** (full form)	21–22	was UC5 State, UC9
Wang Ziang (Eric)	UC3 start, UC4 join	Team creation → **Factory Method**	17	unchanged; UC4 race is a transition strategy
Yu Qianshuo (Yorik)	UC6 refund on expiry	Team outcome consequences → **Observer**	18	was Adapter
Panpakorn (Pan)	Receive notifications	Outbox transports → **Template Method**	19	was UC1 Strategy, UC2
Lim Jiechao	UC8 my orders and teams	Pricing → **Strategy**	20	was UC7 Observer, UC8

EVERY USE CASE HAS AN OWNER · * PROPOSED, NOT YET AGREED

UC1 Lim* **UC2** Pan* **UC3** Eric **UC4** Eric **UC5** Eric* **UC6** Yorik **UC7** Lim* **UC8** Lim **UC9** Liu Jin **Notifications** Pan

[CONFIRM: owners marked * (open decision 4)]

Each member presents their own design problem in section 2; the team statechart (UC4–UC6) goes with Eric's UC4 design.

Report: use case in §3.4, design problem in §3.5 [FILL: section numbers (#463)] [VERIFY: UC9 owner]

JC Lim

Four sprints: the Definition of Done reached, and what each retro changed

Report [FILL: § (#463)] · sprints, DoD, retrospectives

DoD ladder (EPAT 01 Agile p26) coded → tested → integrated → releasable → released

Sprint	Goal	Definition of Done reached by the goal increment
1 · 2–12 Sep	Walking skeleton: sign-in, browse, admin on dev	**integrated**: backend on dev 5 Sep; no frontend, so not releasable
2 · 14–25 Sep	Pay, join a team, see the order, replay-safe	**integrated**: merged, tested, Checkout off on dev; 134 of 150 h
3 · 28 Sep–9 Oct	Invite → pay → refund on deployed dev	**integrated**; goal not met under our “on dev” rule; 117 of 150 h
4 · 12–23 Oct	Complete system on dev; three services on ECS (SWE5001)	[FILL: step as of the 25 Oct rehearsal]

Same basis every sprint: integrated = merged to dev with every check green; releasable = shown working on deployed dev, our “on dev” rule

Retro improvement → change in the next sprint

1 → 2	No burndown, no hours	→ daily board snapshots, dated day logs
2 → 3	A 10,000-line PR; cliff burndown	→ PRs under 1,500 lines, mid-sprint check, “on dev” rule in the DoD
3 → 4	[FILL: retro #451]	→ owners keep board estimates current, checked mid-sprint

Agile artefacts: a story card, a burndown, a practice

Report [FILL: § (#463)] · appendix: agile artefacts

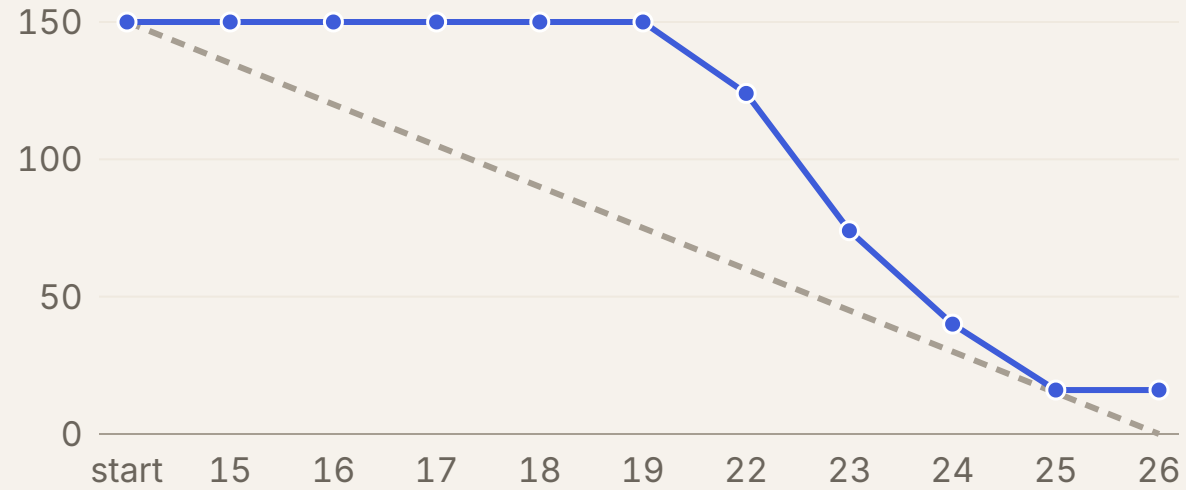
STORY CARD · #221 (US25) — Liu Jin · PR #242

As an admin, I can set a listing's team size, lifetime and team price, so that buyers can start blocs on real listings.

- [x] Admin-only API; 403 non-admin, 401 anonymous
- [x] Size ≥ 2, lifetime in range, team price below individual; 400 with field errors
- [x] Stale expectedVersion → 412; writes audited
- [x] Configured listing is startable; START integration test
- [x] Contract changes additive, merged before the controller
- [x] Unit, PostgreSQL IT, authorization tests; ArchUnit green

Design problem field: State, Optimistic Offline Lock

SPRINT 2 BURNDOWN · HOURS LEFT · REPORT PERIOD 15–26 SEP



Flat for a week, then a cliff: work merged late in large PRs. Reconstructed from merge dates.

Practice (EPAT 03, 05): two red tests reproduced a quote defect (7 Oct log), then PR #424 refactored admission into shared guards and injected the Factory Method's creator. Full set of artefacts: backup B1.

WZ Eric

Logical architecture: COLA layers across six bounded contexts

4+1 LOGICAL VIEW · SAD 04A P24–29

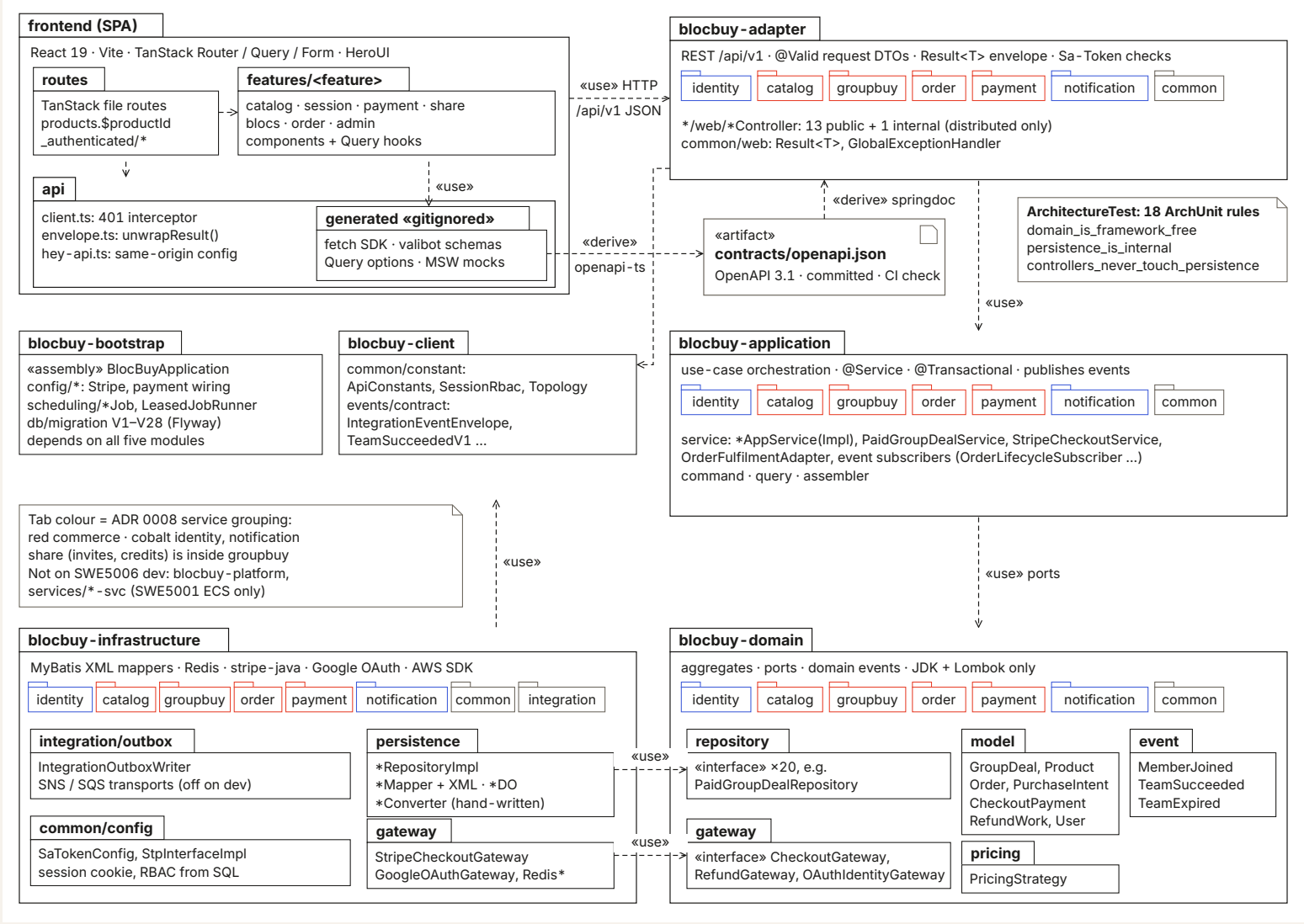
Java 21 · Spring Boot 4 · MyBatis · PostgreSQL 18 · Redis 8 · Sa-Token · Stripe Checkout | React 19 · TanStack Router and Query · TypeScript | AWS EC2, S3, CloudFront

DECISIONS THAT SHAPE IT

Modular monolith, COLA layers, six contexts	pre-ADR D1, D4
Domain framework-free; 18 ArchUnit rules	pre-ADR D1 · no ADR
Consequences run before the commit	ADR 0002
Cross-service events through an outbox	ADR 0007
One codebase, two topologies	ADR 0008
Router owns URL state; generated client	F-ADR 0001 · 0003

pre-ADR = locked decision table D1–D20 (skeleton plan, 30 Aug), written before numbered ADRs

Report §3.1 · Liu Jin, presented by Eric



Transition strategies from analysis to design

Strategy	Concern	Analysis objects · use cases	Become in design	Record
Language, framework	maintainability	«control», «entity» · all	Spring `@Service` by constructor injection; domain stays plain Java	D1–D20, ArchUnit · *no ADR*
Presentation	usability	GUI «boundary» · UC1–UC9	TanStack route + component + Query hook	F-ADR 0001, 0009
Distribution	interoperability	«control» behind a boundary · all	`@RestController` returning `Result`; generated TS client is the stub	D11–D12, F-ADR 0003
Persistence	integrity	«entity» · all	aggregate + repository port ◁ RepositoryImpl, MyBatis mapper, DO	D1, D9 · *no ADR*
External interface	provider isolation	system «boundary» · UC2–UC6	gateway port ◁ Stripe / Google adapter; signed webhook	ADR 0001, 0004, 0005
Security	security	every «control» · all	login interceptor; `checkRole(ADMIN)`; CSRF header; rate limits	D6–D7, F-ADR 0002
Scheduling	expiry ≤ 1 min	Scheduler actor · UC5, UC6	`@Scheduled` job under a database lease, every minute	ADR 0002 · *job: no ADR*
Concurrency	no oversell	«entity» with capacity · UC4	row locks in a fixed order + guarded UPDATE + unique indexes	ADR 0001 · *no ADR*
Domain events	decoupling	«control» → other contexts · UC5–UC7	event record + transactional listener; outbox across services	ADR 0002, 0003, 0007

Template: SAD 04a p37 (object types, use cases, static and dynamic change). Typical class and sequence changes: slide 11.

Report §3.2 · Yorik, Eric

WZ Eric

One strategy pair: analysis objects become design classes

TYPICAL CLASS-DIAGRAM CHANGE

«boundary» → route, hook and generated client · «control» → REST controller and application service · «entity» → aggregate behind a domain repository port, implemented by a MyBatis mapper · external «boundary» → gateway port with an adapter

TYPICAL SEQUENCE-DIAGRAM CHANGE

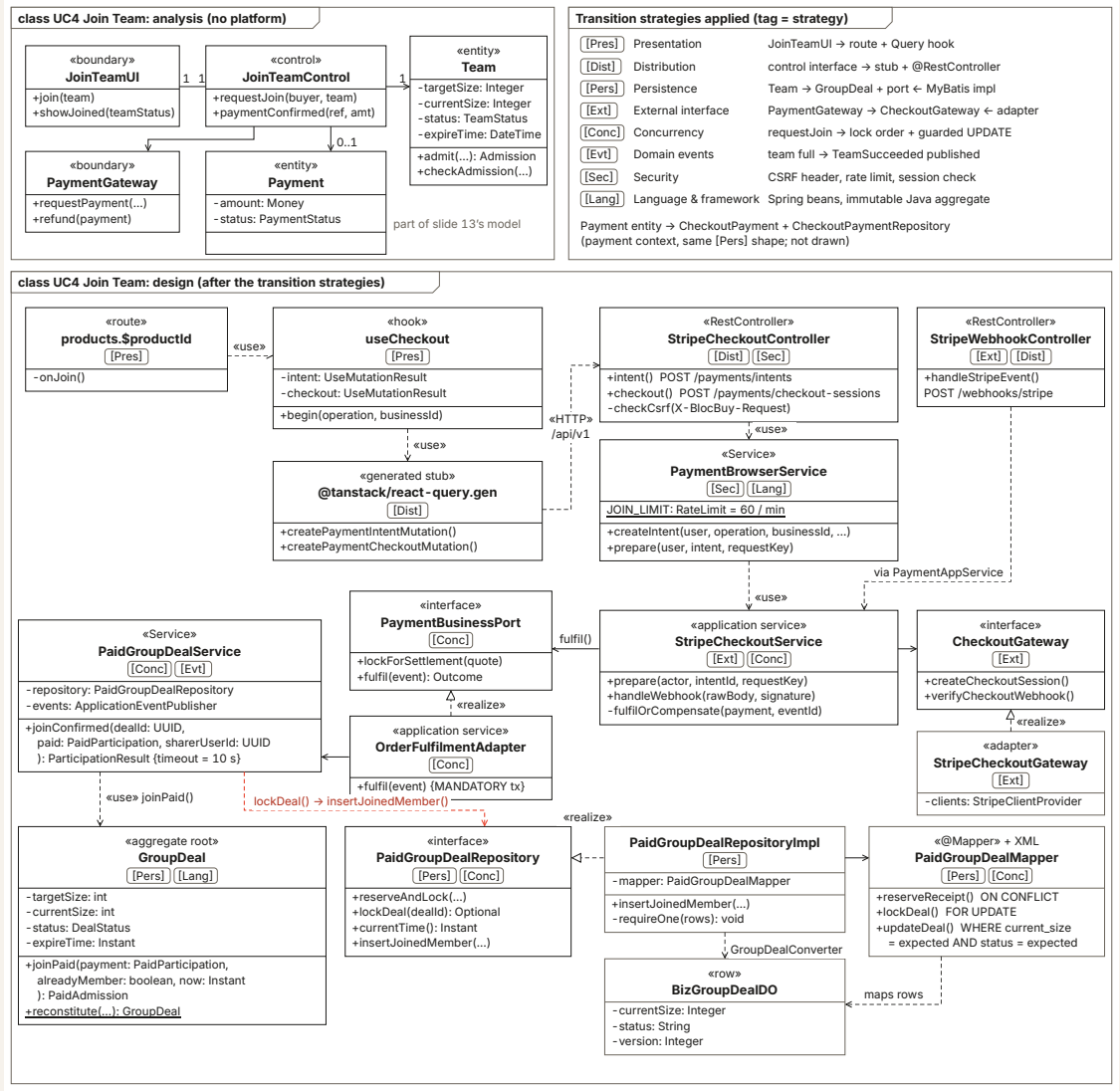
control → entity.update() becomes service → repository.lockDeal() `SELECT … FOR UPDATE` → aggregate.joinPaid() → guarded `UPDATE`, in one transaction

RATIONALE

the domain names its needs as interfaces; infrastructure supplies them (dependency inversion), so persistence and providers can change without touching the rules

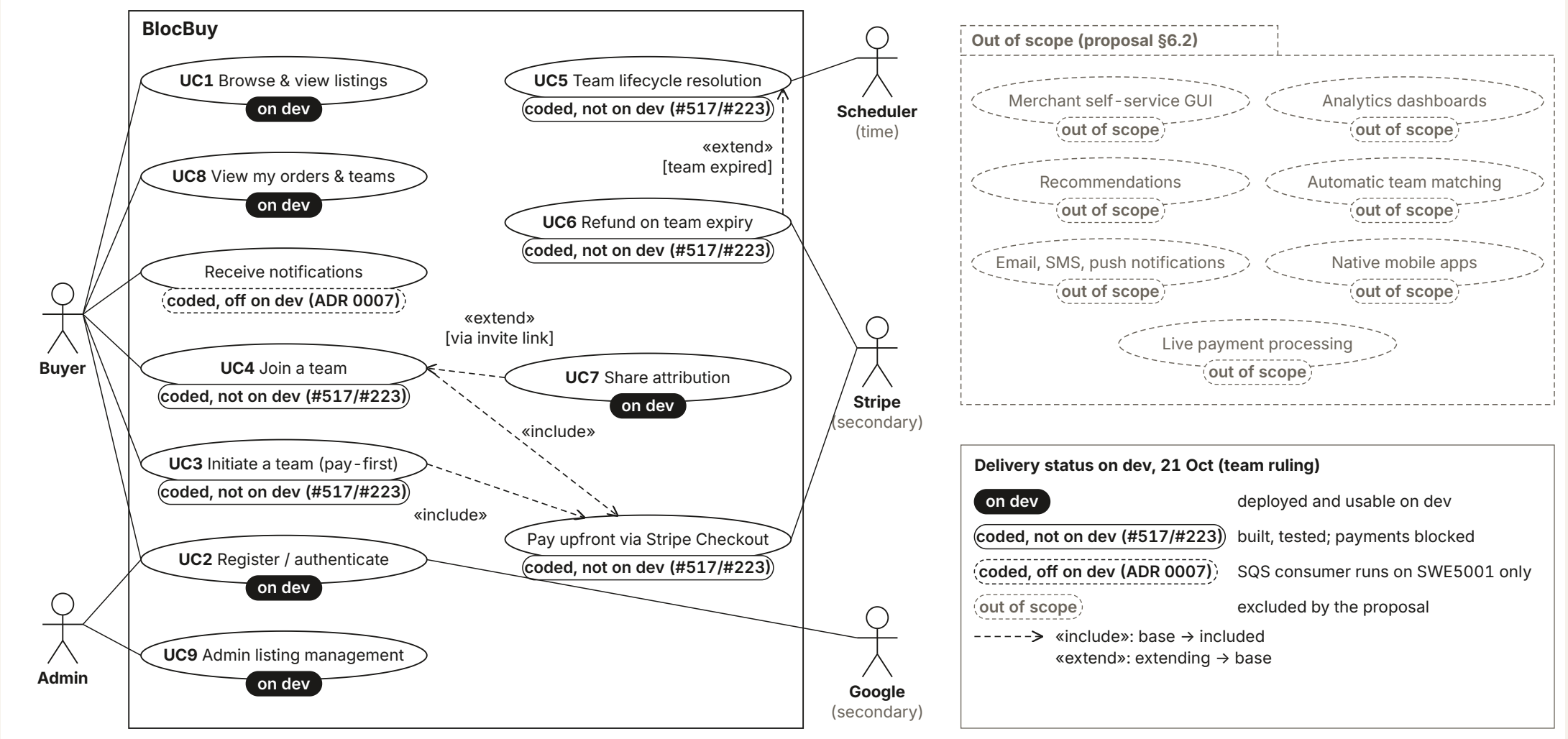
Tags on the design side: Pres, Dist, Pers, Ext, Sec, Conc, Sched, Event (slide 10)

Report §3.2 · transition strategies



WZ Eric

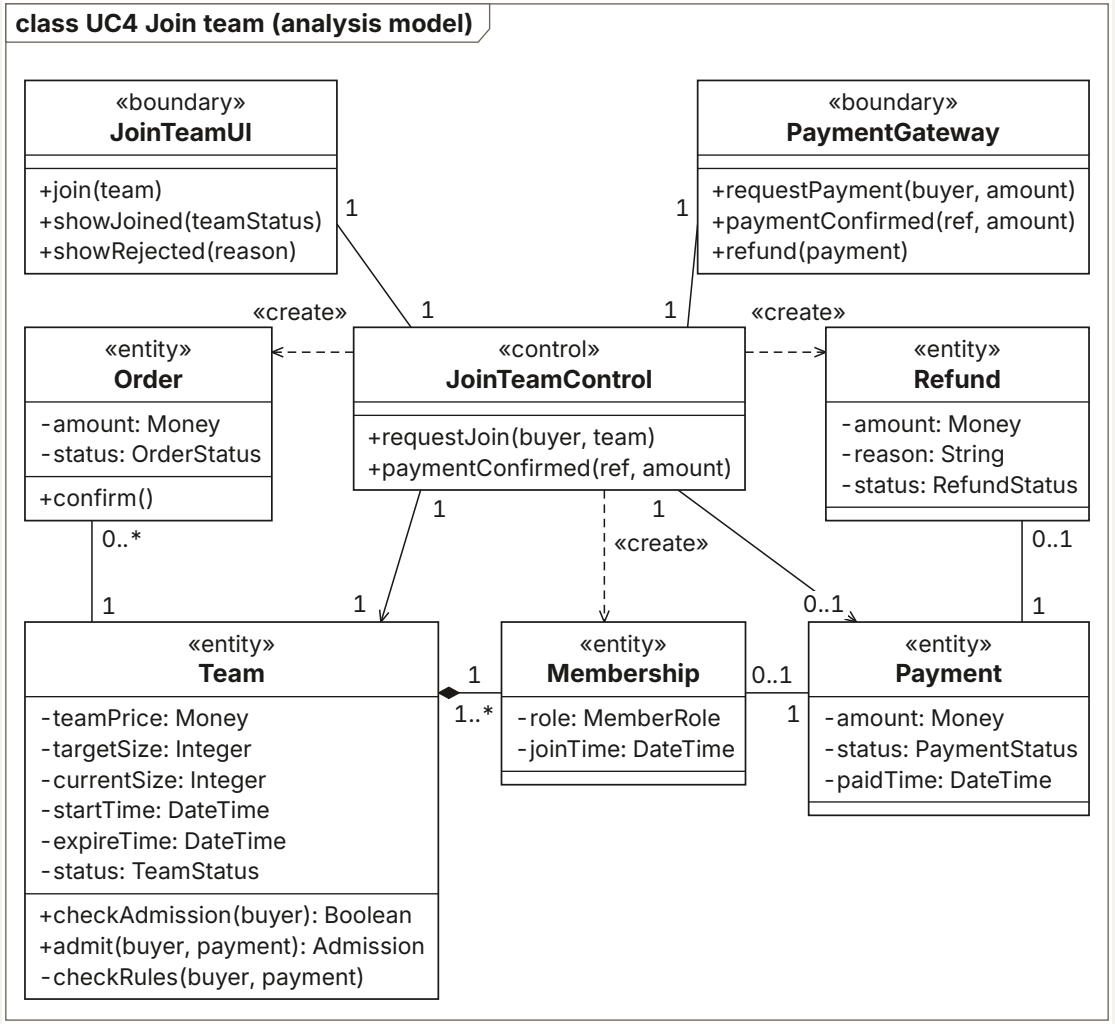
Use case diagram: in scope, out of scope and status

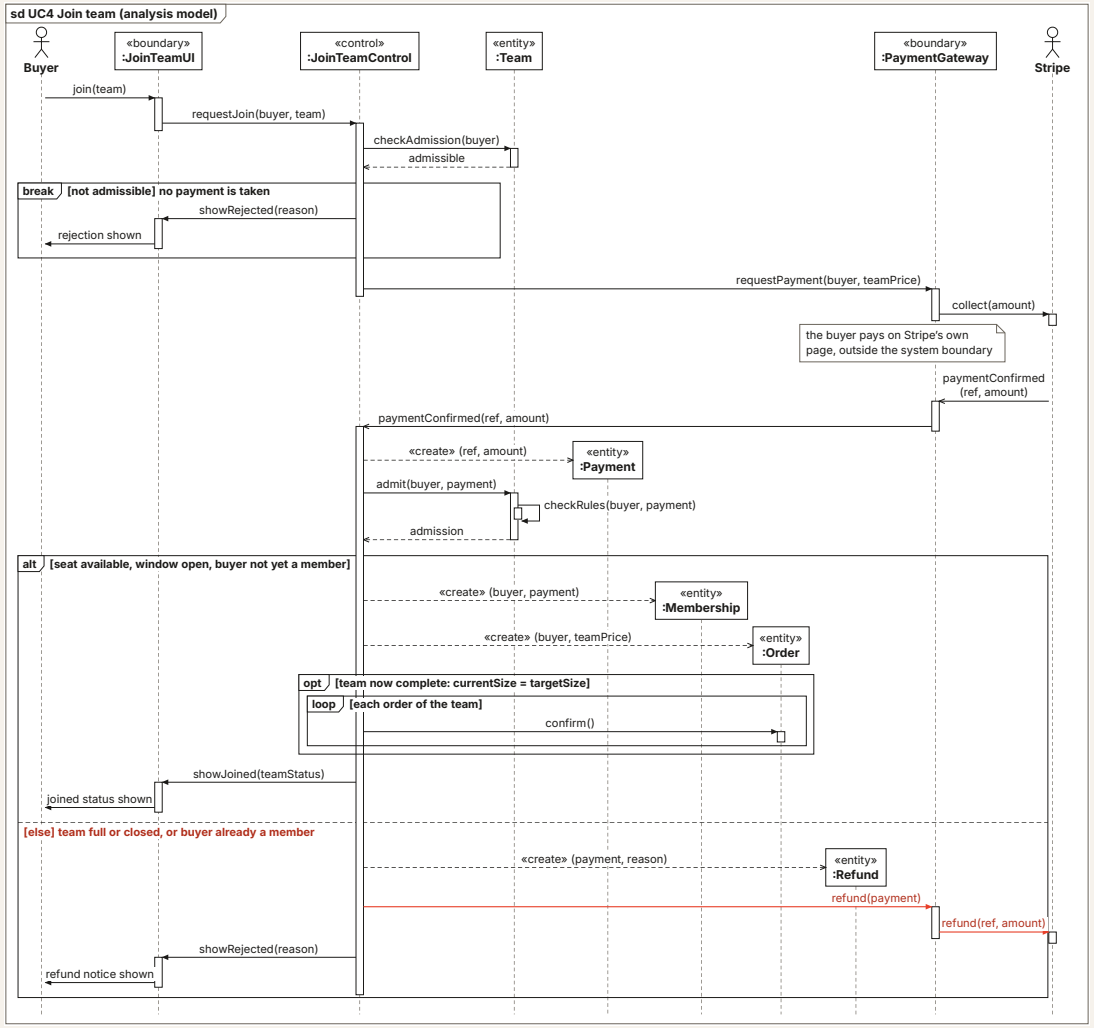


Report §3.3 · use case diagram and specifications

WZ Eric

UC4 join: the analysis model

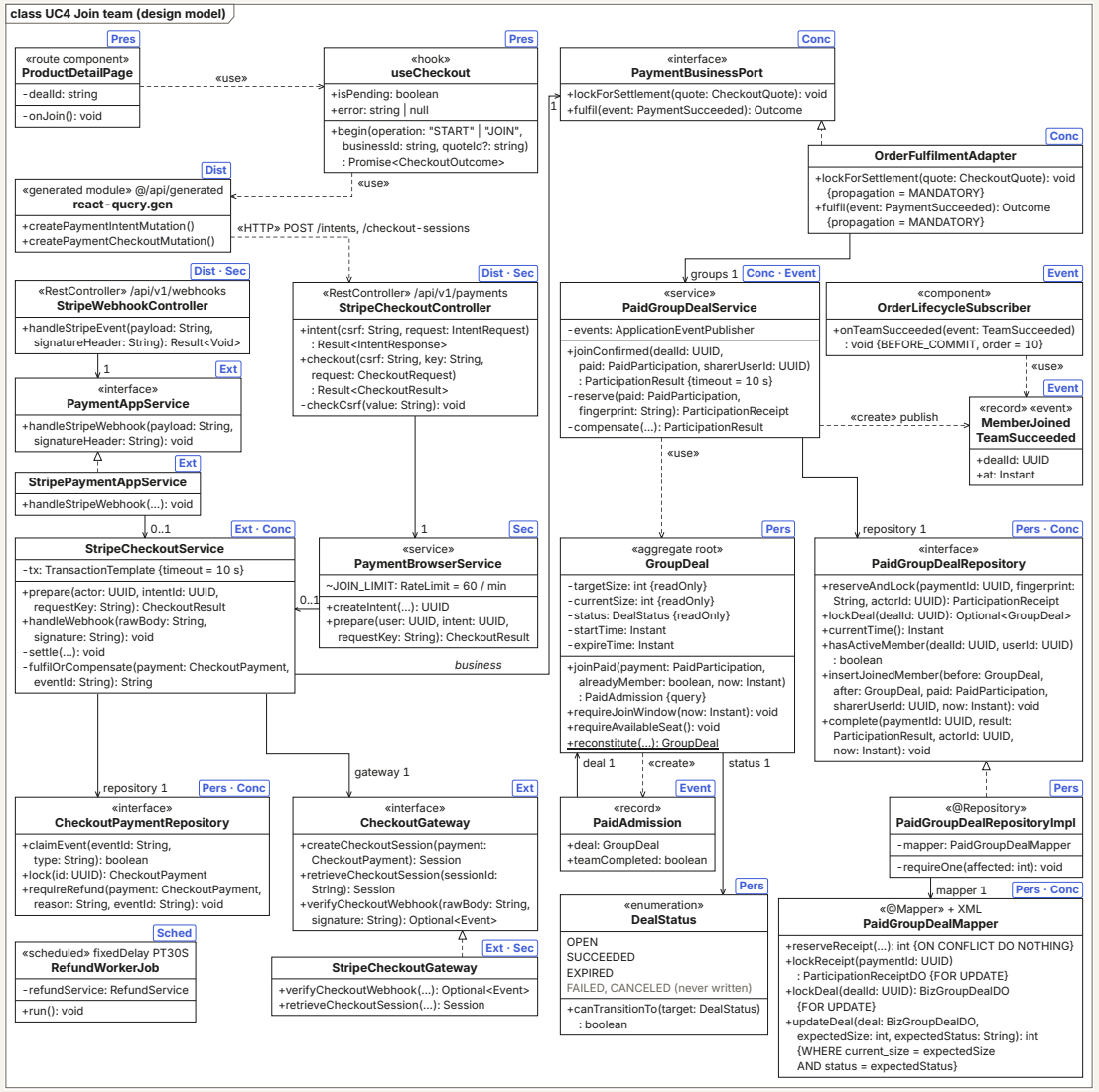




Report §3.4 · Eric (UC4 analysis model)

WZ Eric

UC4 join: the design class diagram



STRATEGY TAGS · SLIDE 10

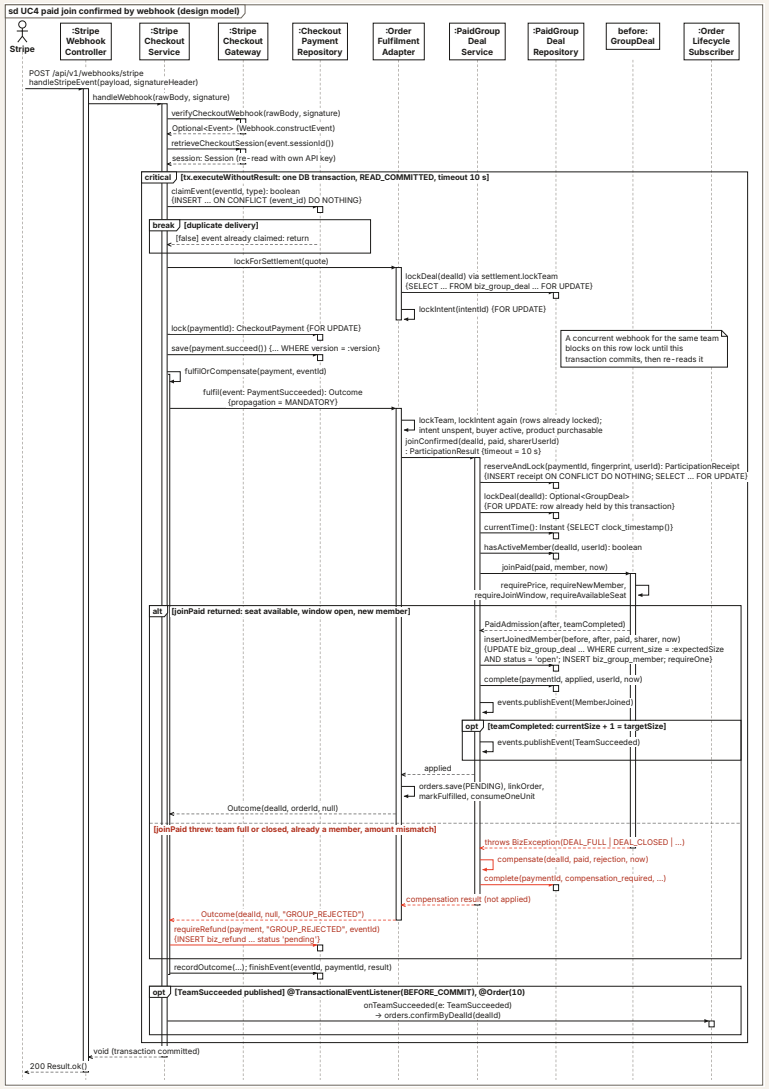
Pres presentation
Dist distribution
Pers persistence
Ext external interface
Sec security
Conc concurrency
Sched scheduling
Event domain events

Typed signatures, visibility and multiplicities: SAD 01b p14–16

Report §3.4 · Eric

WZ Eric

UC4 join: the design sequence, paid join by webhook



MAJOR FLOW

paid join confirmed by the Stripe webhook: signature checked outside the transaction, then one transaction (READ COMMITTED, 10 s), drawn as `critical` because UML has no transaction fragment

CONCURRENCY STRATEGY

lock order team row → intent → payment → receipt, each `SELECT … FOR UPDATE`; a racing join waits on the team row

GUARDED UPDATE

`UPDATE … WHERE current_size = expected AND status = open`; the last seat publishes `TeamSucceeded`

ALTERNATIVE

DEAL_FULL or DEAL_CLOSED: receipt `compensation_required`, a refund obligation, never an extra member

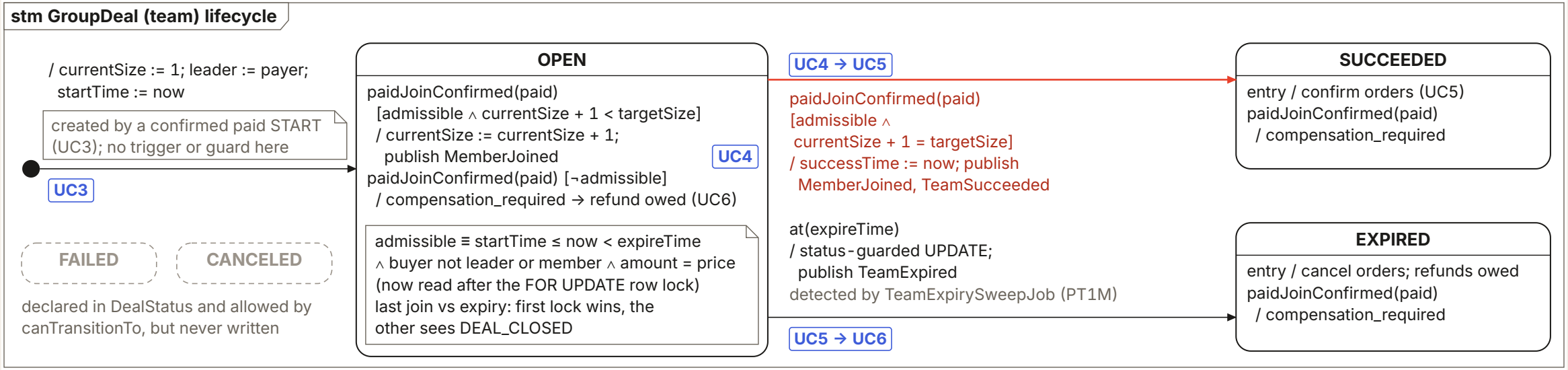
EVIDENCE

`StripeLocalIntegrationTest .twentyBuyersRaceForLastSeat…`: 20 paid joins, 1 seat, 19 refunds owed (local, PostgreSQL in Testcontainers)

Report §3.4 · Eric

WZ Eric

Team-formation statechart



WHY A STATECHART (SAD 03C P4, P11)
team state gates UC4, UC5, UC6 and UC8

GUARDS
`GroupDeal.joinPaid`: price → new member → window → seat

GREYED
FAILED and CANCELED are declared but never written

CODE IT MATCHES
`DealStatus.canTransitionTo`: only OPEN moves, never to itself

TIME EVENT
`at(expireTime)`, detected by `TeamExpirySweepJob` every minute

Report §3.4 · Eric (team statechart)

UC3 start a team: Factory Method

WHERE IT SHOWS (CLASS DIAGRAM)

the «create» of `GroupDeal`: start-team rules must stay fixed while how a team is built may vary; until #424 the Client also named `new StandardPaidGroupDealCreator()`

CAUSE FOR REDESIGN · WHAT VARIES (SDP 05A P15)

creating an object by specifying a class explicitly · the construction step varies while `initiate` stays fixed (GoF p136: a template method calling a factory method)

CANDIDATES

Factory Method · static factory (static "forbids subclassing of the base factory class", SDP 01a p34) · Builder · Abstract Factory

PARTICIPANTS

Creator `PaidGroupDealCreator` (final `initiate` validates, then abstract `createGroupDeal`) · ConcreteCreator `StandardPaidGroupDealCreator` · Product `GroupDeal` · Client `PaidGroupDealService`

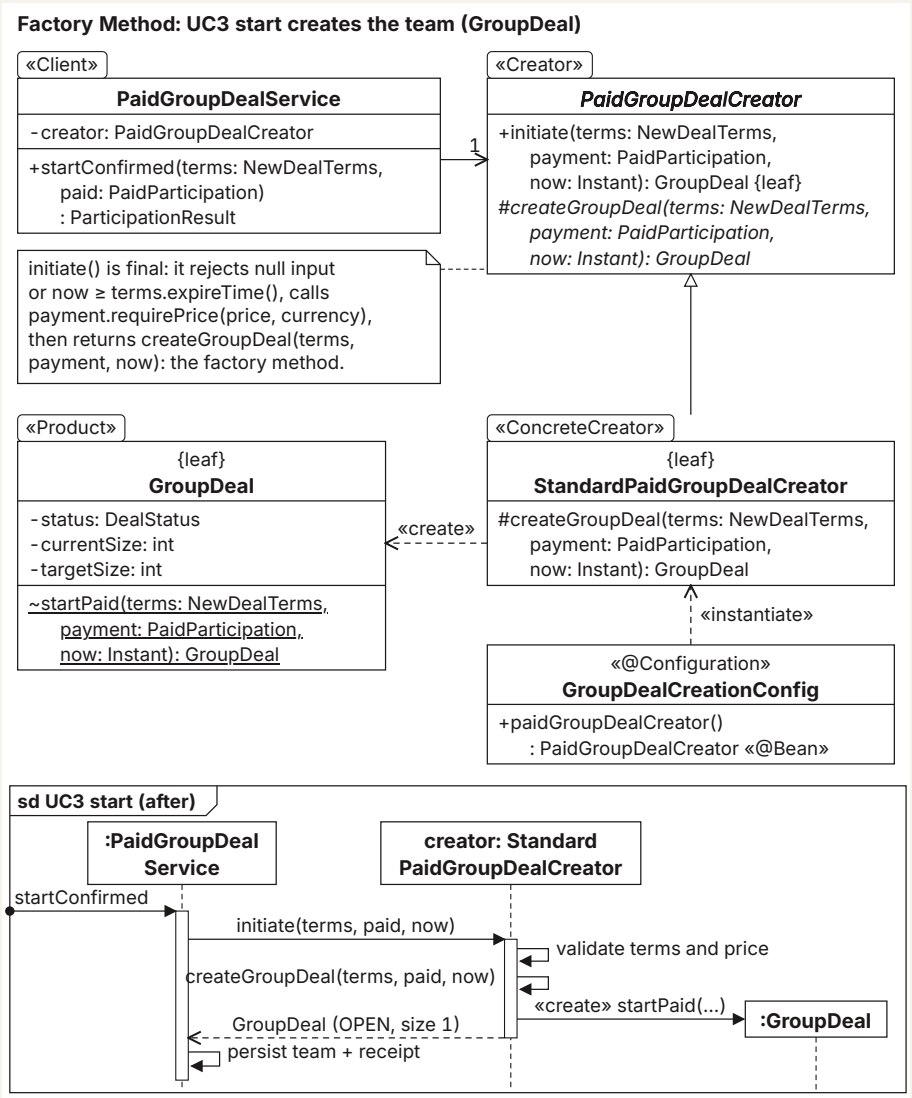
NEEDED BECAUSE · LIABILITY (SDP 05A P35)

every creator keeps the start rules (final initiate); a test or config swaps the creator without editing the service (#421) · one ConcreteCreator today, chosen in config; product class fixed (`GroupDeal` is final)

TESTS

`PaidGroupDealServiceTest.startUsesTheInjectedCreator`

Report §3.5 · Eric (UC3)



YQ Yorik

UC5 to UC6 team outcome: Observer

WHERE IT SHOWS (SEQUENCE)
an expiry that called order and refund code inline: groupbuy would change for every new consequence (ADR 0002)

CAUSE FOR REDESIGN · WHAT VARIES
tight coupling · “number of objects that depend on another object”

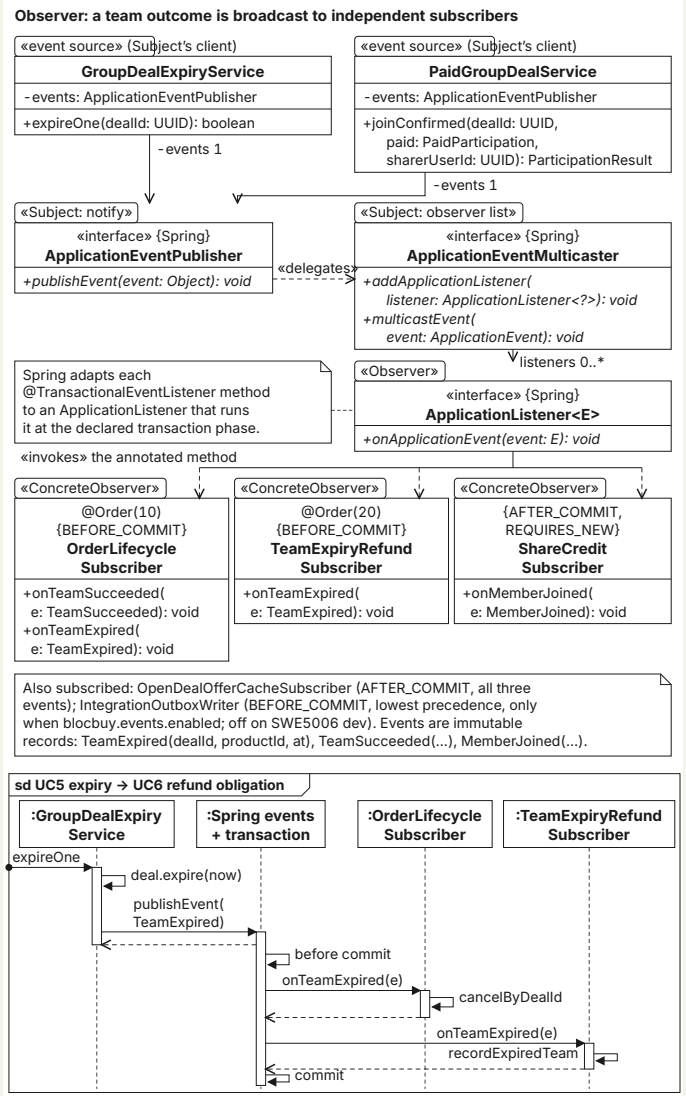
CANDIDATES
Observer · Mediator · direct facade calls · a message broker (ADR 0002)

PARTICIPANTS
Subject (attach, notify): Spring’s `ApplicationEventPublisher` and multicaster · Observer: the listener contract · ConcreteObservers `OrderLifecycleSubscriber` @Order(10), `TeamExpiryRefundSubscriber` @Order(20), before commit · event source `GroupDealExpiryService`

NEEDED BECAUSE · LIABILITY
proposal §6.3: new team consequences without editing domain logic · implicit control flow; a wrong phase is a silent bug

TESTS
`TeamExpiryRefundSubscriberTest` (rolls the expiry back on failure)

Report §3.5 · Yorik (UC6)



Notifications: Template Method for outbox transports

WHERE IT SHOWS (CLASS DIAGRAM)
brute force: the SQS and SNS transports each copy the batch loop, lease checks and result mapping

CAUSE FOR REDESIGN · WHAT VARIES
algorithmic dependencies · “steps of an algorithm”

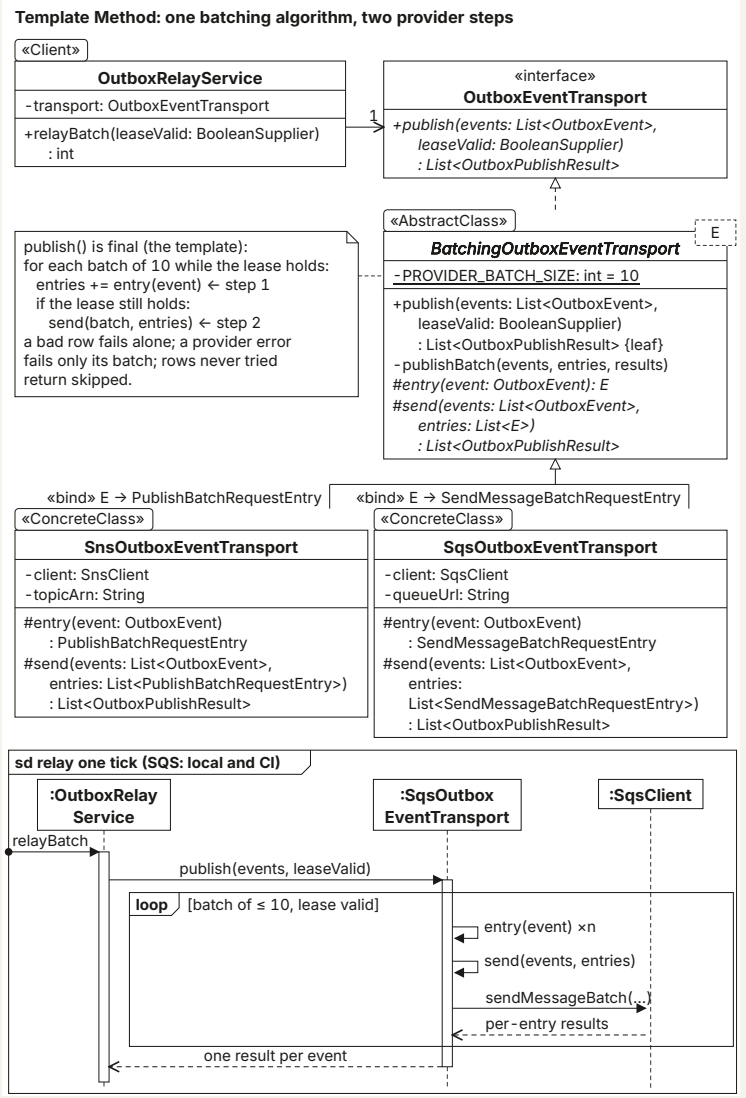
CANDIDATES
Template Method · Strategy (the whole algorithm does not vary) · Adapter per SDK

PARTICIPANTS
AbstractClass `BatchingOutboxEventTransport<E>`, final `publish` · primitive operations `entry`, `send` · ConcreteClasses `Sqs…`, `SnsOutboxEventTransport`

NEEDED BECAUSE · LIABILITY
two providers today: ElasticMQ/SQS locally, SNS on ECS (ADR 0007) · a subclass cannot change the loop

TESTS
`OutboxBatchFailureTest` (6, both providers)

Report §3.5 · Pan (notifications)



UC1 pricing: Strategy

WHERE IT SHOWS (CLASS DIAGRAM)
brute force: browse and the team quote each switch on `pricingMode` with the arithmetic inline

CAUSE FOR REDESIGN · WHAT VARIES
algorithmic dependencies · “an algorithm”

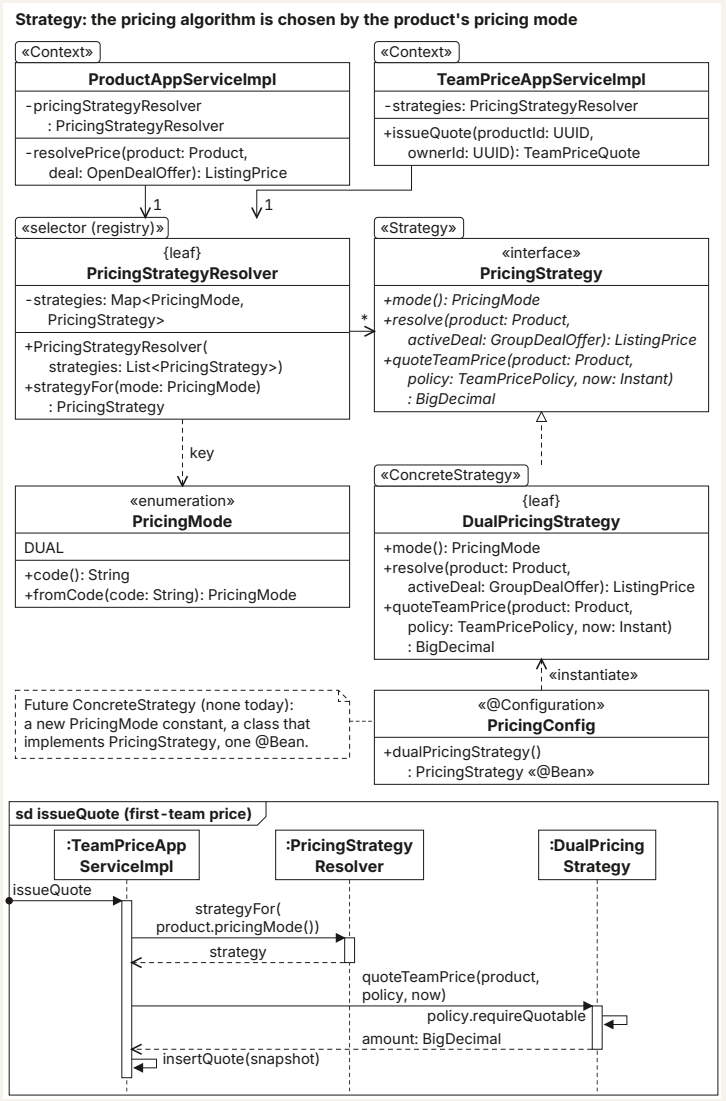
CANDIDATES
Strategy · Template Method (no shared skeleton) · Decorator (no stacking rule)

PARTICIPANTS
Strategy `PricingStrategy` · ConcreteStrategy `DualPricingStrategy` · Contexts `ProductAppServiceImpl`, `TeamPriceAppServiceImpl` · registry `PricingStrategyResolver`

NEEDED BECAUSE · LIABILITY (SDP 05A P35)
proposal §6.3: new pricing schemes are a design driver · one ConcreteStrategy today

TESTS
`PricingStrategyResolverTest` (duplicate mode fails startup), `DualPricingStrategyTest`

Report §3.5 · Lim (UC1)



LJ Liu Jin

Design problem: listing rules that depend on its state

PROBLEM (UC9)

publish, delist, revise and delete are legal in different lifecycle states

WHERE IT SHOWS · BRUTE-FORCE DESIGN (RT 3.5)

the same `switch (status)` in four operations of `Product`; in the sequence, `publish()` branches on status

WHY REDESIGN (SDP 05A P4 WORDING)

hard to maintain: one rule in four places · hard to extend: a new state edits every switch · hard to reuse: the rule cannot be tested apart from Product

WHAT VARIES (SDP 05A P13)

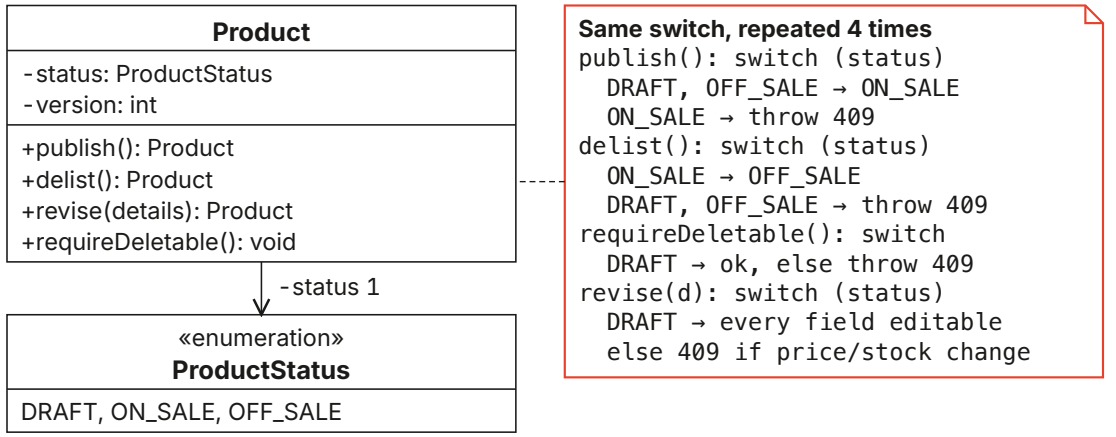
“states of an object”

CANDIDATES

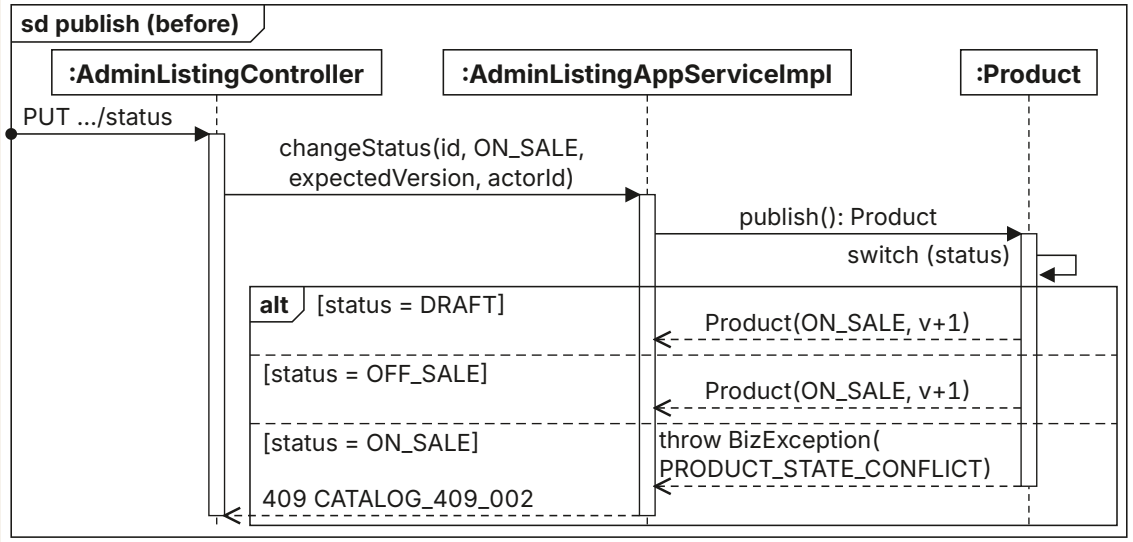
State · Strategy (nobody chooses; the state decides) · transition table, as on the team

Report §3.5 · Liu Jin (UC9)

Before (brute force): a status switch in every operation



Problem: each operation repeats the status test; a new state edits all four.



LJ Liu Jin

After: the State pattern, and why the team uses a table

PARTICIPANTS

Context `Product` · State `ProductStatus` · ConcreteStates `DRAFT`, `ON_SALE`, `OFF_SALE`: enum constants with bodies, each compiled to its own subclass · client `AdminListingAppServiceImpl`

IMPLEMENTATION DECISIONS

the ConcreteState returns its successor and the Context applies it · default bodies refuse with 409 `PRODUCT_STATE_CONFLICT` · stateless, shared states · each transition returns a new revision, version + 1; a stale write gets 412

LIABILITY

a new state still needs a constant, its overrides and tests; the app service keeps one switch that picks the command, not its legality

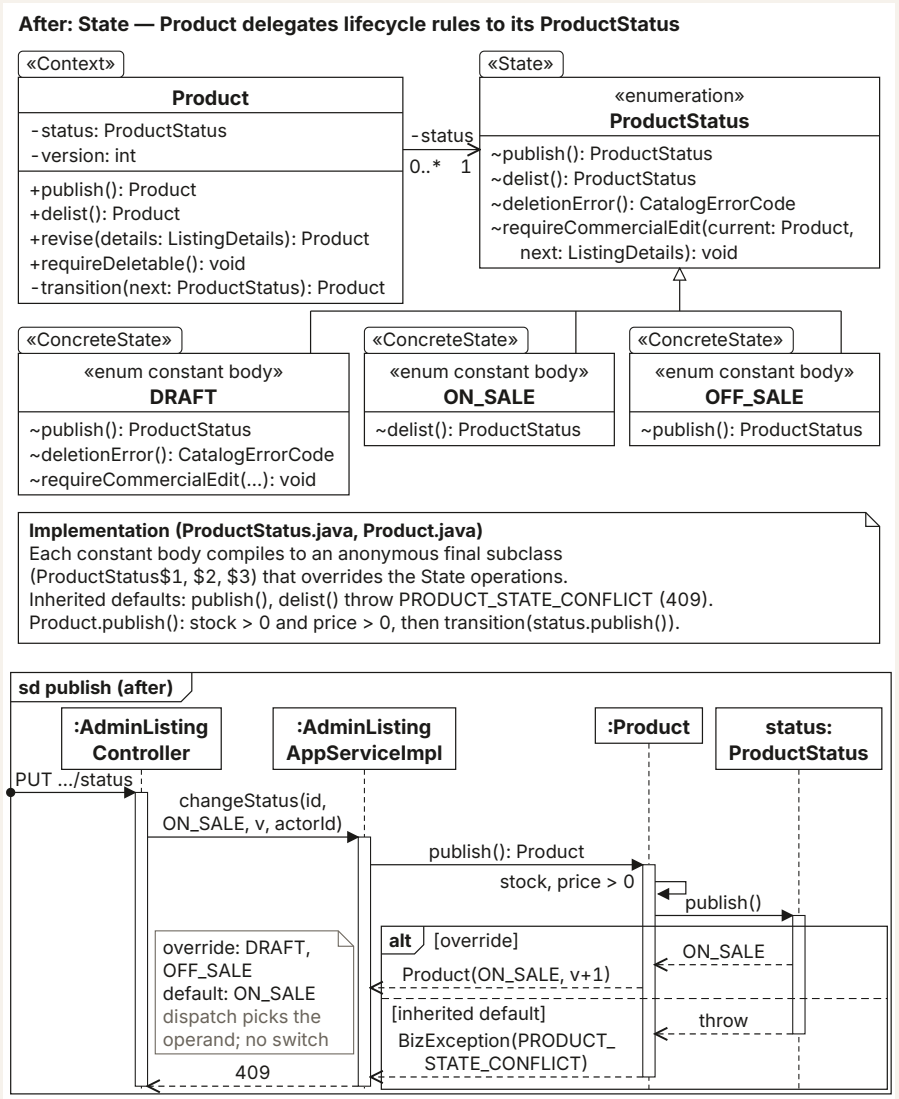
WHY THE TEAM USES A TABLE INSTEAD

only OPEN has behaviour; SUCCEEDED and EXPIRED are final, so `DealStatus.canTransitionTo` suffices (SDP 05a p35). The proposal promised State there: slide 4

TESTS

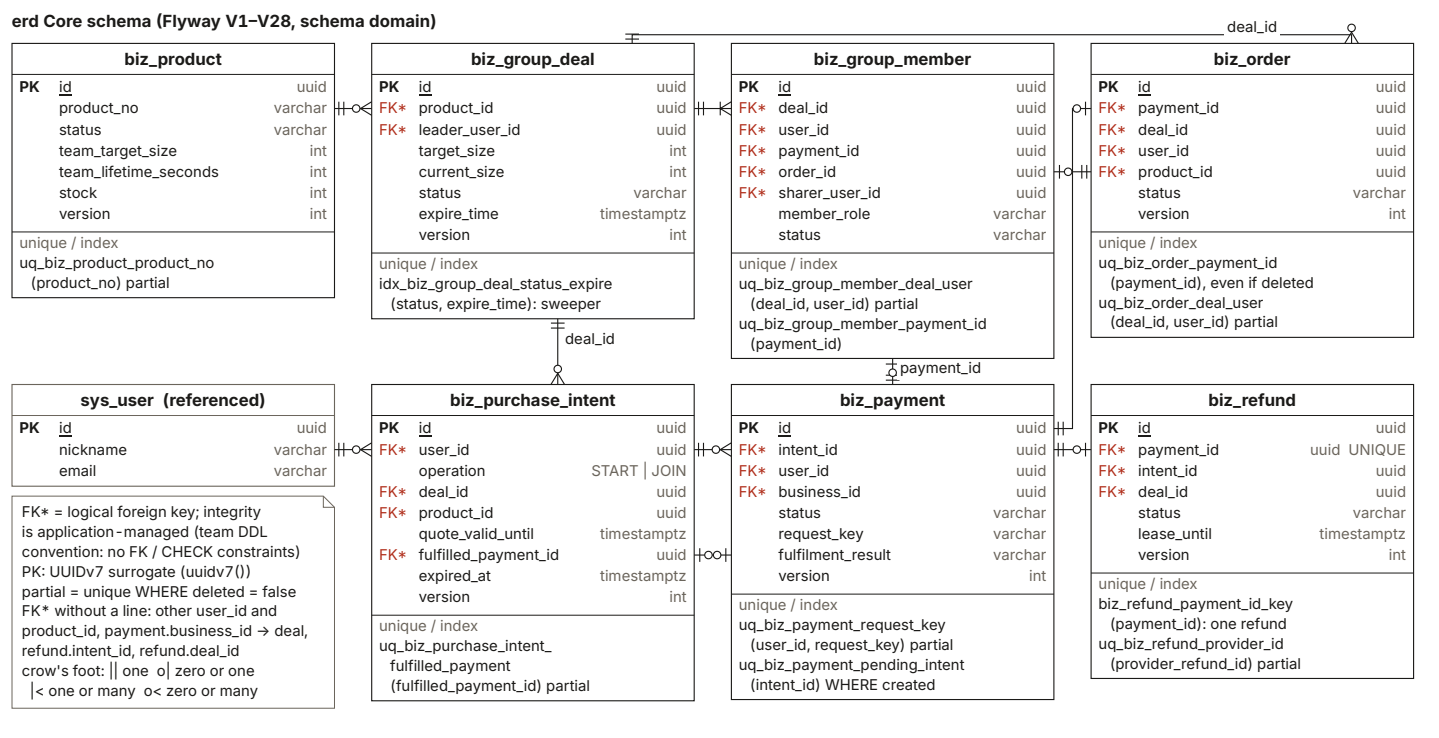
`ProductStatusTest.onSalePublishThrowsStateConflict`, `ProductLifecycleTest`

Report §3.5 · Liu Jin (UC9)



LJ Liu Jin

Database schema: core entities and mapping decisions



MAPPING DECISIONS · SAD 04A P53–61

KEYS

UUIDv7 surrogate primary keys from PostgreSQL 18

ASSOCIATIONS

logical FK columns; integrity is checked by the application. No FK or CHECK constraints is our DDL convention, not a course rule

AGGREGATES

one table per aggregate root; no class hierarchy is persisted, so no inheritance mapping

RACE BACKSTOPS

unique (deal, user) membership · one refund per payment · one order per payment

Report §3.6 · Liu Jin · all tables: B4

LJ Liu Jin

Testing on the Agile Testing Quadrants

Report [FILL: § (#463)] · types of testing

Q2 business-facing · supporting the team — automated

Story acceptance system · `StripeRolloutAcceptanceIT` · real HTTP, scheduler, signed webhooks

End-to-end system · Playwright · 8 specs, on the preview build and on dev

Runs in `build-quality`, `e2e-preview`, `e2e-dev`

Caught: a 502 on e2e-dev during a deploy (3–4 Oct): one host

Q3 business-facing · critiquing the product — manual

Exploratory, acceptance system · by hand, logged

Stripe Sandbox START and JOIN in a browser (20 Sep, local) · sign-in cookie Secure and HttpOnly on dev (8 Oct)

Caught: no defect logged from these runs

Not done: acceptance testing with outside users

Q1 technology-facing · supporting the team — automated

Unit, component unit · JUnit 185 classes · Vitest 77 files

Integration integration · Testcontainers: 45 of 49 classes

Contract, ArchUnit integration · openapi-diff · 18 rules

Runs in `build-quality`, `frontend-check` (required), `contract-check`

Caught: two red tests reproduced the join-quote defect (#422)

Q4 technology-facing · critiquing the product — tools

Performance, load system · k6 on SWE5001 ECS, by hand

Security scanning types · SAST · dependencies (SCA) · image and IaC configuration · secrets

Runs in by hand (load); security gates on slide 28

Caught: full-history gitleaks: 24 matches, 4 badge tokens still open (slide 30)

Complex classes under test (proposal §6.3)

Quadrants: EPAT 03 TDD p7 · levels: p5

Team state machine	`DealStatusTest`, `PaidGroupDealTest`
Last-seat capacity	`StripeLocalIntegrationTest.twentyBuyersRaceForLastSeat…` (integration)
Refund idempotency	`RefundServiceTest.lostResponseRetriesSameDurableKey`

LJ Liu Jin

Scripted performance test: tool, scenario, target

Report [FILL: § (#463)] · performance test

THE SCRIPTED TEST

Tool	k6 1.6.1, image pinned by digest · `perf/k6/browse.js`
Target system	**SWE5001 ECS environment, same code**: CloudFront → ALB → Fargate. Never the shared SWE5006 dev host, which also runs its database
Load	open arrival rate 10 → 200 requests/s in ten 60 s steps; 100 VUs preallocated, 400 max
Mix	7 listing pages : 3 listing details, anonymous; no think time; 10 s timeout
Pass, per step	p95 < 500 ms · errors < 1% · zero dropped requests
Result	run in the system demo (slide 34) · [FILL: 5.3 pre-run label and date]

PROPOSAL TARGET (§6.3)

Interactive operations ≤ 2 s with up to 100 concurrent users

Little's law `N = X · (R + Z)`: 100 users, 0.5 s think, ~30 ms response → **X ≈ 190 requests/s**, inside the 200 driven, with a p95 gate four times stricter than 2 s

The 0.5 s think time is our assumption

EXPIRY ≤ 1 MIN AFTER THE WINDOW

Measured **locally** under 100-user load (30 Sep): 30 teams, worst lag 58.2 s; one cohort waited a full one-minute sweep

On ECS: [FILL: V4 expiry burst, or "local only"]

PS Pan

Source code and artefacts: one trunk, everything traceable to a commit

Report [FILL: § (#463)] · source control strategy · RT 4.1

SOURCE · ONE MONOREPO

`backend/` Maven COLA modules; services/ for SWE5001
`frontend/` React SPA
`contracts/` openapi.json, the API contract
`deployment/` Compose, SSM bundle, deploy.sh
`infra/` Terraform: dev frontend, SWE5001
`perf/ · docs/` k6 · ADRs, sprint logs
`.github/` workflows, CODEOWNERS, Dependabot

BRANCHING · EPAT 07 CI P41–44

Branch by Feature: one short branch per story, a pull request, squash merge onto the trunk **dev**; linear history since 12 Sep. Instead of merging trunk daily, the branch must be up to date to merge.

WHO MAY CHANGE IT

GitHub: a **code owner** approves (backend Liu Jin, frontend Lim, pipelines Pan), six checks green. AWS: IAM Identity Center, one-hour sessions [VERIFY: MFA enforced]. CI: one OIDC role per job, no AWS keys in GitHub.

Exception: two IAM users for SSM port forwarding, one with an access key

ARTEFACTS · EACH TRACEABLE TO A COMMIT

Artefact	Identity	Stored in
Backend image	`git-<sha>-run-<n>`, deployed by `@sha256` digest	ECR, immutable tags, newest 30 kept
API contract	`openapi.json` committed; drift fails CI	repository
TS client	generated from the contract, gitignored	build only (F-ADR 0003)
Frontend bundle	content-hashed files, immutable cache	S3 behind CloudFront
DB schema	Flyway V1–V28	repository
Infrastructure	Terraform plan saved, applied by an operator; CI never applies	S3 state, DynamoDB lock
SBOM	not produced	—

PS Pan

Continuous integration: pull-request jobs and post-merge jobs

Report [FILL: § (#463)] · continuous integration · RT 4.2

1 · On every pull request into dev

trigger: pull_request → dev (ci-backend also on label changes); jobs skip when their paths are untouched

ci-backend.yml

backend/**, contracts/**

build-quality — required
Spotless + Checkstyle
mvnw verify: unit tests, ArchUnit, integration tests, SpotBugs, JaCoCo
OpenAPI candidate from JAR
verified boot JAR

contract-check
openapi.json drift
openapi-diff: breaking

SonarCloud Backend — required
gate: ≥ 80 % new code

docker — required
image ci-<sha>
boot with PG 18 + Redis 8.10; /api/actuator/health UP within 60 s;
Trivy image scan

ci-frontend.yml

frontend/**

frontend-check — required
pnpm check: gen:api, tsc, oxlint, oxfmt, Vitest, knip; build dist/

SonarCloud Frontend — required
≥ 80 % new code

e2e-preview
Playwright on the PR's dist/

security.yml

gitleaks — required
PR commits

workflow-lint
actionlint

trivy-deps
Maven + pnpm

trivy-config
IaC, Docker

deployment-bundle
ShellCheck; Compose must hold only the backend service; SSM bundle file allowlist and size

Other PR workflows, path-filtered

typesafe-smoke (live search contract) · perf-tooling (k6 tooling)
infra-swe5001: terraform test, Checkov, Trivy config (SWE5001 IaC)

Approval gate

ruleset dev-require-review
· 1 approval incl.
· code-owner review
· stale approvals dismissed
· threads resolved
· branch up to date
· 6 required checks green
· squash merge only

2 · After squash merge: push to dev

automatic deploy to dev (not production), no manual step

backend/**, deployment/backend/** changed and ENABLE_EC2_DEPLOY = true

package-dev
mvnw package -DskipTests:
merged commit

publish-ecr-dev
OIDC → ECR dev/blocbuy
tag git-<sha>-run-<n>
digest checked in ECR

deploy-existing-ec2 · environment backend-dev · OIDC

pre-check
current health

SSM Run Command
deploy.sh pulls @sha256; on-host health fails → automatic rollback

public health
via CloudFront

frontend/**, contracts/** changed and ENABLE_S3_DEPLOY = true

package-dev
pnpm build:
dist/ from the merged commit

deploy-s3-dev
env frontend-dev, OIDC
hashed assets immutable, index.html no-cache + CloudFront invalidation

e2e-dev
Playwright smoke against the dev CloudFront URL

Also on push to dev
SonarCloud dev baseline (backend, frontend)
gitleaks (security.yml)
weekly, Mon 02:17 UTC: full security.yml

PS Pan

Security controls by pipeline stage, and what each one catches

Report [FILL: § (#463)] · pipeline security · RT 4.2

Stage	Control	Catches	Gap
Pre-commit	Lefthook: format, lint, type-check (frontend)	style and type errors	—
Commit · PR	**gitleaks** required on the PR's commits, full history weekly; code-owner review; actionlint	secrets, unsafe workflows	—
Build	SAST: **SonarCloud** required, SpotBugs; SCA: Trivy deps, Dependabot daily	code flaws, vulnerable libraries	CodeQL off: code scanning unavailable on this repository
Test	**Trivy image** required (HIGH, CRITICAL); Trivy config, deployment-bundle rules	image CVEs, IaC and Compose misconfiguration	no DAST against the running site
Deploy	OIDC role per job; image by digest; read-only container, all capabilities dropped	stolen keys, swapped images	rebuilt after merge, not rescanned (ECR scans on push)

Types tested (slide 24, Q4): static analysis, dependency (SCA), image and IaC configuration, secrets. Every third-party action is pinned to a commit SHA. Stages: DSOEA 07 slide 3.

PS Pan

Delivery: one environment, the code owner approves, the deploy verifies itself

Report [FILL: § (#463)] · continuous delivery · RT 4.3

ENVIRONMENTS

one: **dev**. The SWE5006 prod stack was planned, not applied (#226)

APPROVER

the **code owner** on the pull request; the merge releases

PRACTICE

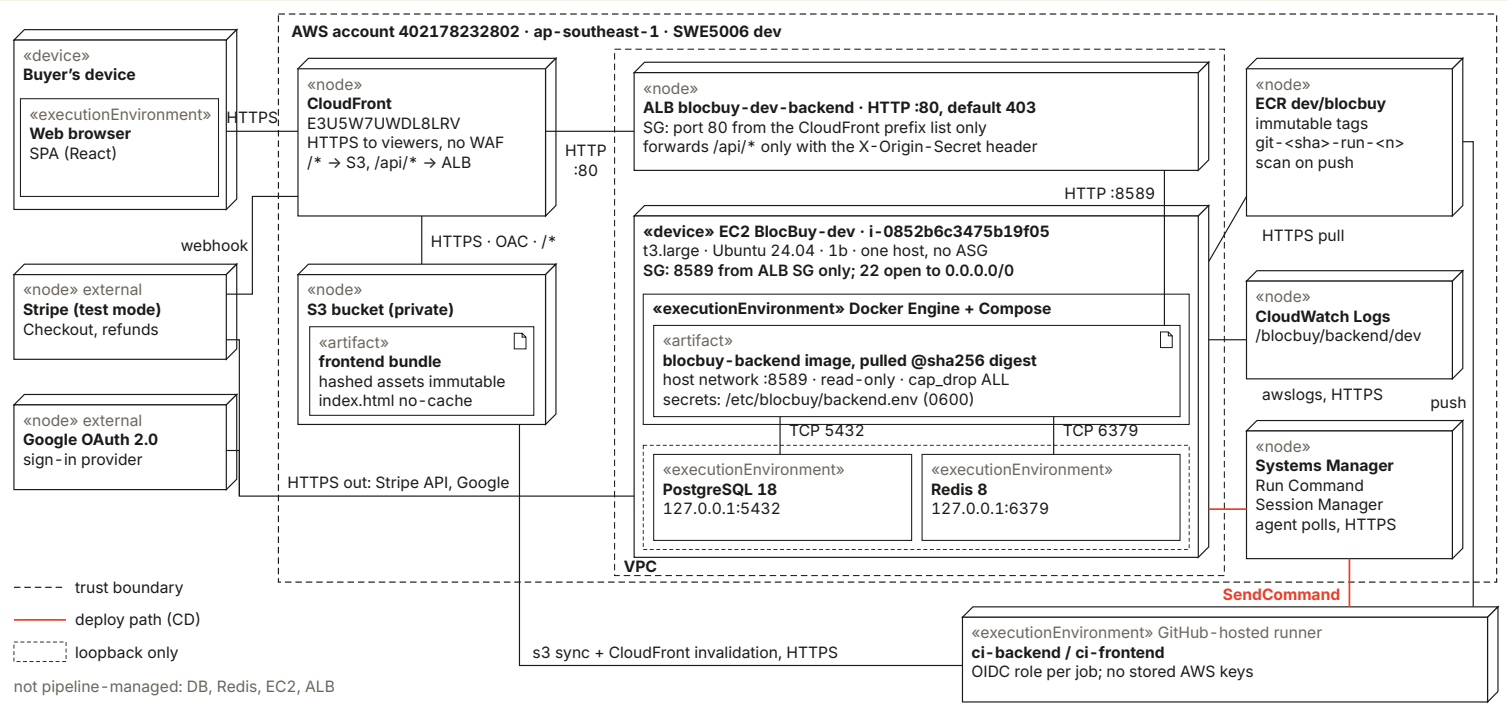
every merge deploys automatically to **dev**, not production; no production, so no promotion gate. SWE5001 prod waits for an approved promotion PR: **Continuous Delivery** (EPAT 07 CI p34–35)

VERIFIED BY

on-host health check with **automatic rollback**; public health via CloudFront; Playwright smoke on dev

TRADE-OFF

PostgreSQL and Redis on the app host: cheap and always on, one point of failure



PS Pan

Technical findings: three issues, their root causes and fixes

Report [FILL: § (#463)] · technical findings

Finding	Root cause	Fix
First dev deploys failed (14–17 Sep)	Hand-built environment drifted from its design: OIDC trust used the old subject format; the deploy policy kept a placeholder instance ID; port 8589 was opened wide	Fixed on dev by hand; the prod stack renders each value in Terraform, so a plan shows it. No deploy failed after 17 Sep (to 19 Oct)
build-quality red for two days (19–20 Oct)	Dependabot upgraded the SpotBugs engine without its Maven plugin; the plugin lacked the engine's BCEL classes	Upgrade both together, one Dependabot group, a POM policy test (PR #580)
Weekly full-history gitleaks failing (since 21 Sep)	20 matches were SHA-256 file digests in a contract baseline, not secrets	Each digest re-hashed, then fingerprinted exactly (PR #580); 4 badge-token findings still open

Still open: the Stripe webhook never confirms a payment on dev (#517), so a green pipeline is not yet a working paid flow. Sources: backend-prod-terraform-deployment.md, ci-failure-audit-20261021.md.

PS Pan

Live: one change from pull request to dev

Step	What you see	Time
P1	The pull request: one frontend change, six required checks green	60 s
P2	Lim approves as the frontend code owner	30 s
P3	Squash merge: one commit on dev	20 s
P4	Delivery runs; the backend PR and deploy, triggered beforehand	90 s
P5	Verified: e2e-dev green, the change live on dev	90 s
P6	Merge to live in minutes; hand-over	40 s

WHY THE BACKEND IS PRE-RUN

Backend PR checks take about 16 min, longer than the segment, so that run was triggered before the talk and we walk through it finished.

BACKUP VIDEO

Recorded on dev by 24 Oct, one clip per step, in the cloud and on a USB drive. `[FILL: link]`

YQ Yorik

System demo: two buyers, one bloc, and the qualities it shows

Step	What you see	Use cases	Quality attribute (slide 3)	Runs on
D1	Leader signs in, starts a bloc, pays in Stripe Sandbox, copies the invite	UC2 · UC1 · UC3 · UC7	security: only a signed webhook confirms a payment	dev, if GO
D2	Joiner opens the invite, joins and pays: 2 of 2, the team succeeds	UC4 · UC5 · UC7	correctness: guards re-checked after payment	dev, if GO
D3	My blocs and my orders, filtered by the URL	UC8	functionality	dev
D4	The bloc started at 16:05 expired and refunded itself	UC5 · UC6	unattended expiry and refund, recorded once; ≤ 1 min measured locally (slide 34)	dev, if GO
D5	No session, a buyer on admin, a write without the marker	—	security: 401 · 403 · 400 · HttpOnly	dev
D6	Twenty paid buyers race for the last seat	UC4	correctness under concurrency	local test
34	Scripted k6 load, pre-run	UC1	performance: ≤ 2 s at 100 users	SWE5001 ECS
36	Back to the §4 pipeline run	—	maintainability	dev

“dev, if GO”: live on deployed dev after the 23 Oct go/no-go (#517, #223); otherwise the 24 Oct recording of that step, said as recorded. A local result is never presented as a dev one.

YQ Yorik

Live: two buyers, one bloc

Step	What you see	Time
D1	Start a bloc and share the invite	120 s
D2	A friend joins and pays: the team succeeds	120 s
D3	My blocs and my orders	75 s
D4	The 16:05 bloc: expired, order cancelled, refunded	90 s
D5	Security: 401, 403, 400, HttpOnly cookie	60 s
D6	Last-seat race: the test, in the IDE	45 s

TWO BUYERS, ONE SCREEN

Two Chrome profiles, Leader and Joiner, both signed in with Google. Stripe Sandbox: a test card, no money moves.

STARTED BEFORE THE TALK

A team of two on `US34-04` at 16:05: its 40-minute window, the shortest allowed, ends ≈ 16:45.

BACKUP VIDEO

Recorded on dev by 24 Oct, one clip per step, in the cloud and on a USB drive. `[FILL: link]`

YQ Yorik

Performance against the target: the scripted k6 run

K6 PRE-RUN · SWE5001 ECS ENVIRONMENT, SAME CODE · NOT THE DEV HOST

Requests/s	p95 (ms)	Errors	Dropped	Step
10	—	—	—	—
20	—	—	—	—
40	—	—	—	—
60	—	—	—	—
80	—	—	—	—
100	—	—	—	—
125	—	—	—	—
150	—	—	—	—
175 ●	—	—	—	—
200 ●	—	—	—	—

● at or above ≈ 190 requests/s, the proposal's 100 users (slide 25) · results [FILL: from the 24 Oct pre-run]

TARGET (PROPOSAL §6.3)

≤ 2 s with up to 100 concurrent users

Our gate per step: p95 < 500 ms, errors < 1 %, no dropped requests

Verdict: [FILL: pass or fail per gate]

EXPIRY ≤ 1 MIN

Sweep every minute under a lease; worst lag 58.2 s for 30 teams under 100-user load (**local**, 30 Sep)

NOT CLAIMED

Authenticated paths under load on a deployed system; anything about the dev host; scaling (one task)

YQ Yorik

Scalability: a stated trade-off, not a claim

DEV, THE SYSTEM YOU SAW

Compute	one EC2 t3.large, no Auto Scaling group
Data	PostgreSQL 18 and Redis 8 on the same host
Effect	one point of failure; a deploy restarts the API
Read	AWS, read-only, 21 Oct

WHY, DELIBERATELY

- one complete system, always on, at low cost
- the proposal lists no scalability target (§6.3)
- one codebase, two topologies (backend ADR 0008)

Scale path, not claimed here: the same code on ECS in our SWE5001 topology, commerce 2–6 tasks on requests per target, notification 1–4 on queue backlog (backup B10).

YQ Yorik

Maintainability: the pipeline you just saw, and the seams in the code

SHOWN IN §4 (SLIDE 31)

One change, live on dev: six required checks, a code-owner review, an automatic deploy and a smoke test. 18 ArchUnit rules hold the module boundaries on every build.

PROPOSAL §6.3: THE PRIMARY DESIGN DRIVER

Change	Added as	Pattern
New pricing mode	one more `PricingStrategy` and its `PricingMode` constant	Strategy · slide 20
New consequence of a team's outcome	one more subscriber to `TeamSucceeded` / `TeamExpired`	Observer · slide 18
New payment provider	one more adapter behind `CheckoutGateway`, `RefundGateway`	gateway port, external-interface strategy · slide 11

Existing rules stay unedited: each change adds classes; a pricing mode also adds one enum constant.

Thank you

Questions?

github.com/PPsyrius/iss-groupbuy-team8 · d1b3np88z657cp.cloudfront.net

Backup slides: index

B1 **Agile artefacts appendix**
Where are the backlog, stories, sprint plans, reviews and retros?

B2 **Sprint 2 and 3 burndowns**
How did earlier sprints burn down?

B3 **ADR index**
Which decisions are recorded, and where?

B4 **Database schema, all tables**
Which tables does each context own?

B5 **Expiry and refund sequence**
How does an expired team become refunds?

B6 **UC6 refund recovery: candidates and decisions**
How do you never refund twice?

B7 **Test inventory**
How many tests of each kind, and where do they run?

B8 **Workflows, required checks and timings**
What runs when, how long, and how often is it green?

B9 **Known gaps and limitations**
What is not done yet in the pipeline and on dev?

B10 **SWE5001 production topology**
What would scaling out look like?

B11 **Stripe Sandbox refund, local evidence (20 Sep)**
Is the refund real, and where has it been proven?

B12 **Scale-out and load results (SWE5001 ECS + local baseline)**
How does it scale, and how fast is it?

B13 **UC9 listing statechart**
Which listing moves are legal in each state?

B14 **UC4 design diagrams, complete**
Can we see every class and message of the join?

B15 **Every member's use case and design problem**
Where are the other members' models?

B16 **Entity-relationship diagram, all tables**
What does the whole schema look like?

Click a row (or type its number in the URL as #B6). Slide 37 stays up during questions; open a backup only when a question needs it.

JC Lim

Agile artefacts appendix

Artefact	Where it lives	State on 21 Oct
Product backlog	GitHub Issues and Project board 2 (GroupBuy Kanban Board); issue templates in .github/ISSUE_TEMPLATE	170 items: 137 Done, 15 Ready, 15 Backlog, 3 In progress
User stories with acceptance criteria	UC issues #24–#32; story issues (e.g. #221, #223, #180); AC per story in docs/agile/sprint-03/plan.md §5	template: complexity, accountable members, design problem, AC
Sprint plans	S1 #49; S2 Sprint 2 plan (docx) and #93; S3 docs/agile/sprint-03/plan.md and #232; S4 docs/agile/sprint-04/plan.md and #465	all four sprints
Burndown charts	S2 chart in docs/sprint2-progress-report.typ; daily data docs/agile/metrics/*.csv (board-snapshot.yml, since 17 Sep)	S2, S3 reconstructed (B2); S4 from snapshots
Sprint reviews	S1, S2 in the progress reports; S3 docs/agile/sprint-03/review.md	S4 due 23 Oct `[FILL: S4 review]`
Retrospectives	S1 report; S2 report §Sprint Retrospective	S3 missing (#451) · S4 due 23 Oct `[FILL: S3, S4 retros]`
Day logs, DoD	docs/agile/sprint-0N/log/ (S2–S4); DoD in the S2 report, sprint-03/plan.md §8, sprint-04/plan.md §8	dated logs per working day

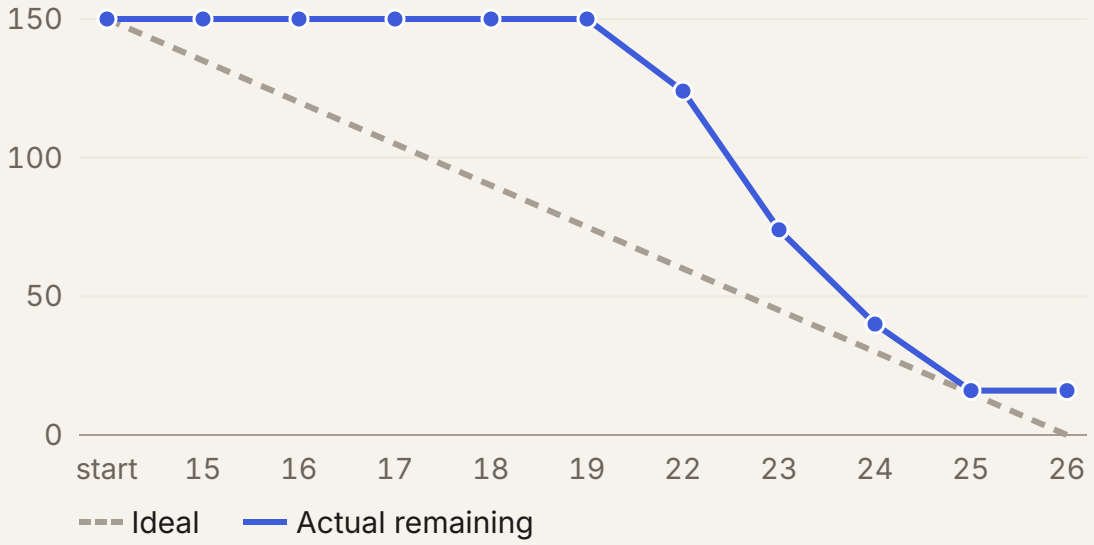
Source: board and issues read with gh on 21 Oct; docs/agile/README.md. The report appendix holds the full set (#463).

JC Lim

Sprint 2 and 3 burndowns

Sprint 2 · remaining hours, 15–26 Sep

150 h planned · 16 h left · reconstructed from merge dates

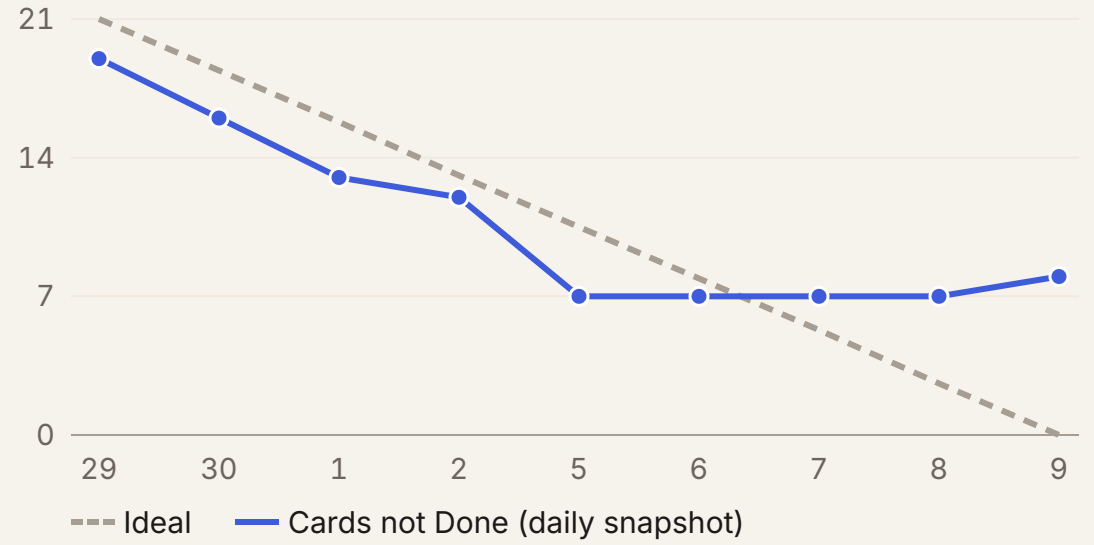


Flat for a week, then a cliff: work merged late in very large pull requests. Retro answer: PRs under 1,500 lines and hours on the board daily.

Source: docs/sprint2-progress-report.typ, "Daily burndown data"

Sprint 3 · open sprint cards, 29 Sep–9 Oct

21 cards · 150 h planned · 33 h left at freeze (review)



Steadier burn, then stuck at 7 from 5 Oct: #223, #224 and #229 waited for Checkout on dev, #226 for the NFR-37 decision. Hours come from the review.

Source: docs/agile/metrics/.csv, sprint-03/review.md*

WZ Eric

ADR index

Backend · backend/docs/adr

#	Decision
0001	Purchase intent lives in order; group terms on the product
0002	Team resolution is published; consequences run before commit
0003	Referral credit is a post-commit subscriber
0004	Retire the unused legacy PaymentIntent path
0005	Gate Stripe Checkout with a deployment property
0006	Payment owns the admin attention read model
0007	Outbox, one SNS topic, one SQS queue per consumer (Proposed)
0008	One codebase, two topologies: dev monolith, three services on ECS

Frontend · frontend/docs/adr

#	Decision
0001	React SPA, pure CSR; Router owns URL state, Query server state
0002	Same-origin /api with an HttpOnly cookie session
0003	Generated API client, not committed
0004	One build artifact promoted dev to prod
0005	SPA fallback by a CloudFront Function
0006	oxlint and oxfmt toolchain
0007	Hand-managed dev CloudFront, then Terraform import
0008	Documentation and evidence conventions
0009	Result envelope: one unwrapResult as select
0010	React Compiler via the Babel preset
0011	Testing strategy: Vitest, MSW, Playwright smoke
0012	Agent configuration in AGENTS.md
0013	pnpm and Node pinned, exact versions
0014	Stripe Hosted Checkout with a server-owned intent
0015	Admin console shell, roles guard, versioned writes

LJ Liu Jin

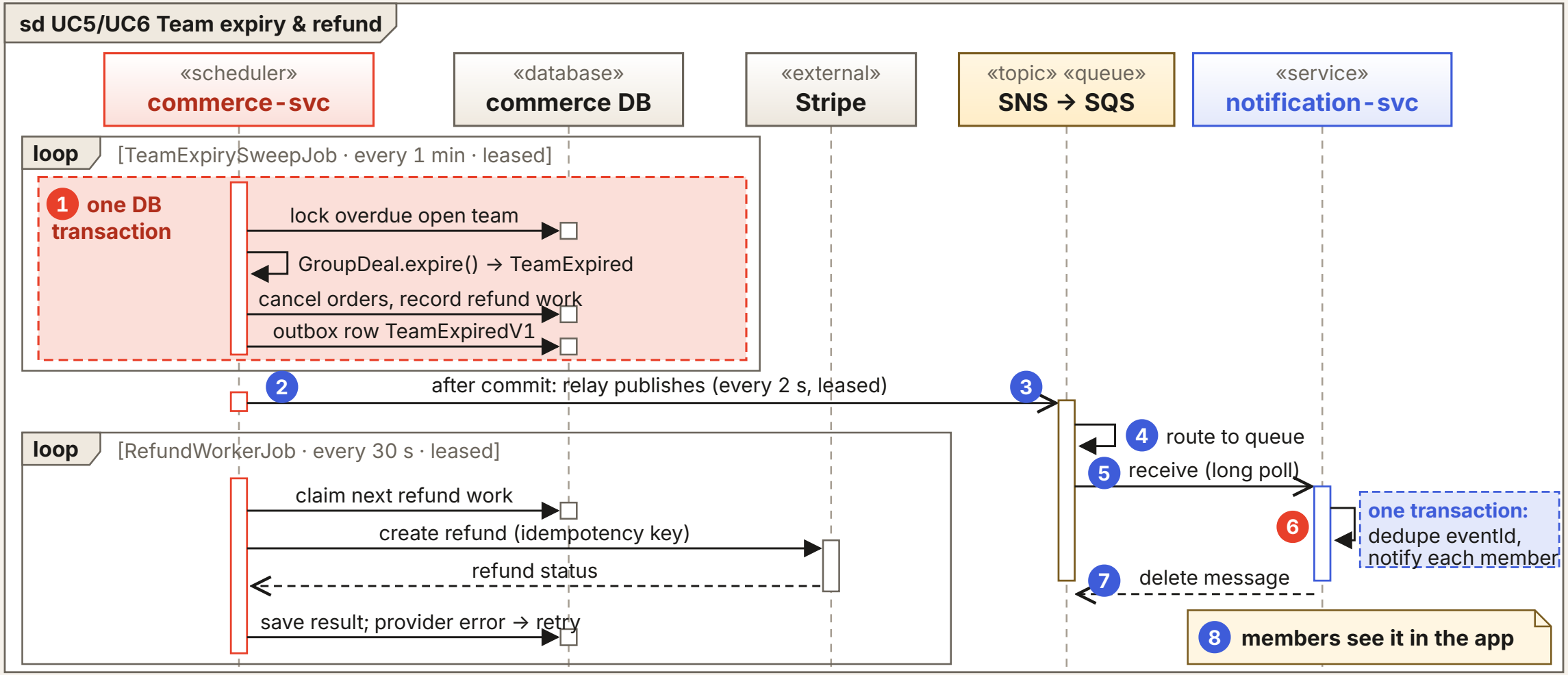
Database schema, all tables

Context	Tables	Migrations	Keys that make “once” hold
identity	`sys_user, sys_user_oauth, sys_role, sys_permission, rel_user_role, rel_role_permission`	V3, V24	one binding per Google subject
catalog	`biz_product, biz_team_price_policy, biz_team_price_quote, log_product_team_terms`	V4, V6, V14, V20, V21	biz_product.version (Optimistic Offline Lock)
groupbuy	`biz_group_deal, biz_group_member, biz_group_participation_receipt, biz_share_invite, biz_share_credit`	V5, V7, V18	one member per user and team; one receipt per payment
order	`biz_order, biz_purchase_intent`	V12, V15, V19, V22	intent fulfilled by one payment
payment	`biz_payment, log_payment_webhook_event, biz_refund, log_payment_review, biz_payment_review_case, log_refund_attempt`	V2, V9–V11, V16	webhook event_id; one refund per payment
notification	`biz_notification, sys_consumed_message`	V27, V28	PK (event_id, consumer); (event_id, user_id)
jobs, outbox	`sys_outbox, sys_scheduler_lock, sys_scheduled_item_retry, sys_checkout_expiry_cursor`	V13, V17, V25, V26	one lease per job; outbox id = eventId

28 Flyway files, 29 tables in schema `domain`; V1 sets the rules: snake_case, timestamptz, numeric money, invariants in the database. On ECS (ADR 0008): one database per service on one RDS instance; identity-svc and notification-svc own their rows above, commerce-svc the rest.

YQ Yorik

Expiry and refund sequence



As built for ECS. The SNS → SQS hop and notification-svc (steps 2–8) are the ECS topology; on dev the monolith runs no broker.

YQ Yorik

UC6 refund recovery: candidates and implementation decisions

Candidate	Decision	Why
Mark refunded, then call Stripe	**Rejected**	A crash would report money returned that never left
Hold a DB transaction across the call	**Rejected**	A rollback cannot undo a refund; locks held over network I/O
Idempotency Key per obligation	**Selected**	Every retry reuses one request identity
Retry after a delay	**Selected**	Fixed 30 s in code, not exponential backoff
Human-in-the-loop review	**Selected**	Mismatched or ambiguous work leaves automatic claims
Distributed transaction with Stripe	**Rejected**	No shared protocol across PostgreSQL and the provider

Implementation decisions

1. Obligation commits with the expiry, before any I/O
2. Key from the obligation ID: `"blocbuy-refund-" + id`
3. Short claim and finish transactions; lease and version fence workers
4. Verify payment facts before, provider snapshot after
5. After 23 h: search Stripe by metadata, never create
6. Reconcile only retrieves an existing refund; no admin endpoint exists

Source: docs/swe5006/report-drafts/yorick-yu-contribution.md. Evidence is local with a provider substitute.

LJ Liu Jin

Test inventory

Kind · level	Tools	Count by module	Example classes	Runs
Unit · unit	JUnit Jupiter, Mockito	185 *Test classes, 1,329 test annotations: domain 42, application 41, infrastructure 47, bootstrap 46, adapter 4, platform 2, commerce-svc 2, client 1 (some, like StripeCheckoutFlowTest, use real PostgreSQL)	`RefundServiceTest`, `PaidGroupDealTest`, `DealStatusTest`, `ProductStatusTest`	every PR (build-quality, required)
Integration · integration	Testcontainers: PostgreSQL 18, Redis 8.10, ElasticMQ	49 *IT classes (45 start containers), 242 annotations: bootstrap 24, infrastructure 15, services 9, platform 1	`OrderRepositoryIT`, `TeamExpirySchedulingIT`, `StripeRolloutAcceptanceIT` (acceptance)	every PR (build-quality, required)
Architecture · unit	ArchUnit	18 rules in 1 class	`ArchitectureTest`	every PR (build-quality, required)
API contract · integration	springdoc export, diff of contracts/, openapi-diff	1 contract file	`contracts/openapi.json`	backend PRs (contract-check, not required)
Component · unit	Vitest, Testing Library, MSW	77 files: features 58, routes 10, components 4, test 3, api 2	`account.test.tsx`, `admin-listing-edit.test.tsx`	every PR (frontend-check, required)
End-to-end · system	Playwright, Chromium	8 specs, 24 test declarations	`buyer-journey.spec.ts`, `join-refusal.spec.ts`, `smoke.spec.ts`	PR (e2e-preview); after deploy (e2e-dev)
Performance · system	k6 1.6.1 (SWE5001 ECS); Node workload (local)	k6 browse V2/V3; 3 local scenarios	`perf/k6/browse.js`, `backend/perf/workload.mjs`	by hand (k6 tooling tested in CI)

Counted 22 Oct at a1c6ac6e: 186 `*Test.java` (185 + ArchitectureTest), 49 `*IT.java` (45 start containers). Latest full runs: backend 2,048 passed, 1 opt-in skip (21 Oct log, local); frontend 799 (19 Oct log).

PS Pan

Workflows, required checks and timings

Workflow · trigger	Jobs (required in blue)	Median	Green
ci-backend · PR	**required**: build-quality (13.1 min), docker (2.8 min), SonarCloud Backend; contract-check (12 s)	16.1 min to docker end (18 runs, 16–19 Oct)	18 of 20 finished runs; 16 cancelled by a newer push
ci-backend · push to dev	package-dev 46 s → publish-ecr-dev 73 s → deploy-existing-ec2 40 s; SonarCloud Backend refreshes the dashboard	3.1 min to deployed (88 runs)	deploy 88 of 97; all 9 failures 14–17 Sep (SSM: instance not in a valid state)
ci-frontend · PR	**required**: frontend-check (3.4 min), SonarCloud Frontend; e2e-preview (69 s), intent-stale	4.8 min to e2e-preview end (19 runs)	19 of 20 finished runs
ci-frontend · push to dev	package-dev 35 s → deploy-s3-dev 21 s → e2e-dev 82 s; SonarCloud Frontend	2.7 min to e2e-dev green (94 runs)	deploy 97 of 97; e2e-dev 94 of 97 (3–4 Oct)
security · PR, dev, weekly	**required**: gitleaks (10 s); trivy-deps (72 s), trivy-config, workflow-lint, deployment-bundle when their paths change	12 s on most PRs	PR 97 of 98 finished; dev push 100 of 100
rollback-backend · manual	validate, then SSM rollback to the recorded previous digest or a given one	—	never run
SWE5001 · prod branch	swe5001-release-gate, swe5001-deploy, infra-swe5001 (promotion PR; see the SWE5001 deck)	—	—

Last 100 runs per workflow and event, read with `gh run list` and the jobs API on 19 Oct. PR durations count only runs where the gated jobs ran (docs-only PRs skip them in about 10 s). Push medians run from the push run start, not the merge click.

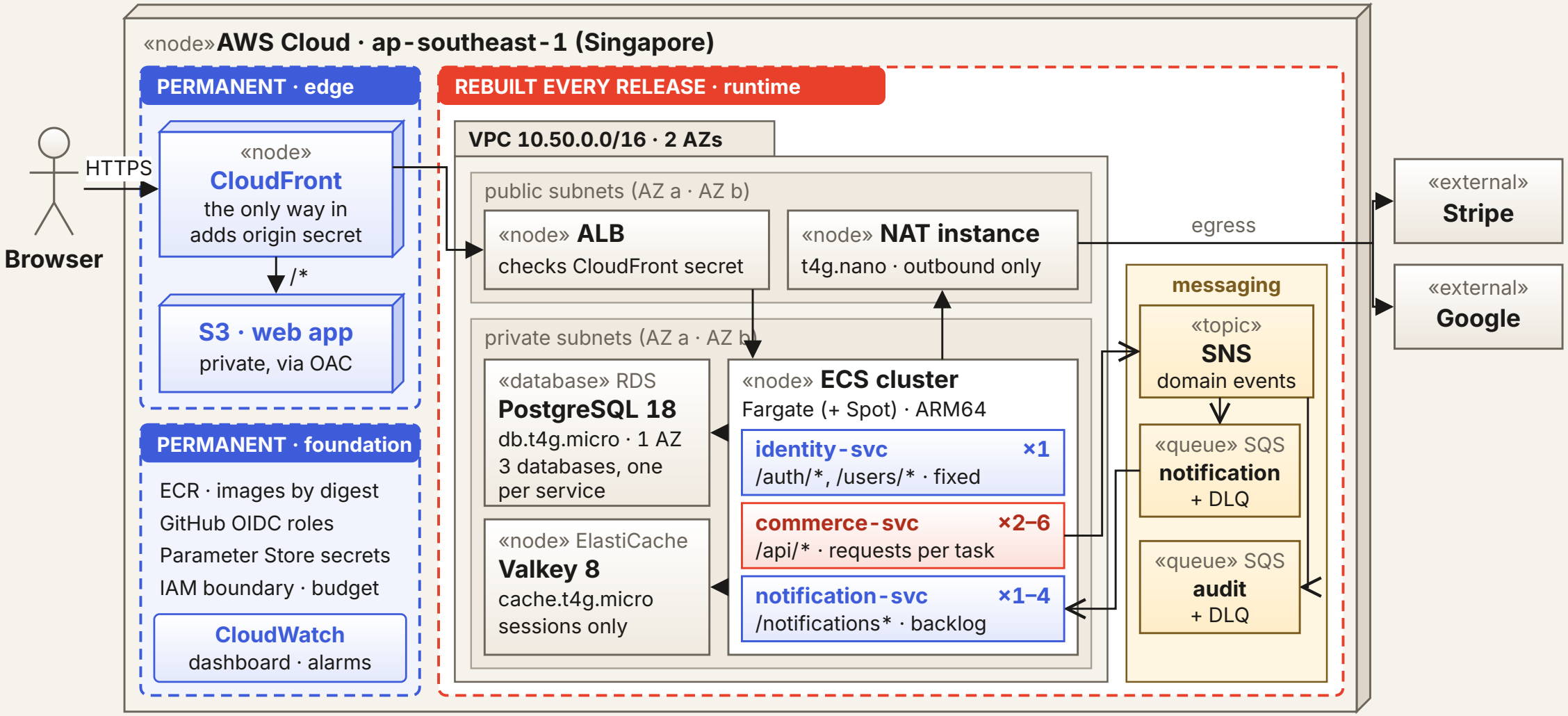
PS Pan

Known gaps and limitations

Gap	Effect	Status
CodeQL off	no CodeQL SAST; SonarCloud and SpotBugs remain	ENABLE_CODEQL=false; code scanning not enabled on the repository
CloudFront → ALB over HTTP	/api payloads and the origin header cross the AWS network unencrypted	NFR-37 TLS still unmet; accepted risk on dev until 16 Nov (#227, NFR register); fix needs an origin hostname and ACM
SSH open to the internet	port 22 from 0.0.0.0/0 on the dev host, which has a public IP; SSM sessions make it unnecessary	AWS read-only check, 21 Oct; open item 9
Dev EBS volume not encrypted	database and host data at rest unencrypted	AWS read-only check, 19 Oct; not raised as an issue yet
Rollback never exercised	recovery path untested on the real host	rollback-backend.yml: 0 runs; no automatic rollback has fired
No dev alarms	failures are noticed by e2e-dev or by people	all six CloudWatch alarms belong to SWE5001
Dev not zero-downtime	a deploy restarts the only API container (502 on 3–4 Oct)	accepted for dev
Secrets in a host file	manual creation and rotation, root-only 0600	accepted for dev; prod uses Parameter Store
Rebuilt after merge	the deployed image and dist/ are not the PR-scanned artefacts; Trivy not re-run	strict up-to-date checks; ECR scans on push; ADR 0004 text to reconcile
Weekly full-history gitleaks red	20 digest false positives fingerprinted; 4 badge-token findings still fail the scan	PR #580, ci-failure-audit-20261021.md
No DAST, no SBOM	no scan of the running site; no component inventory per image	not planned before 26 Oct

PS Pan

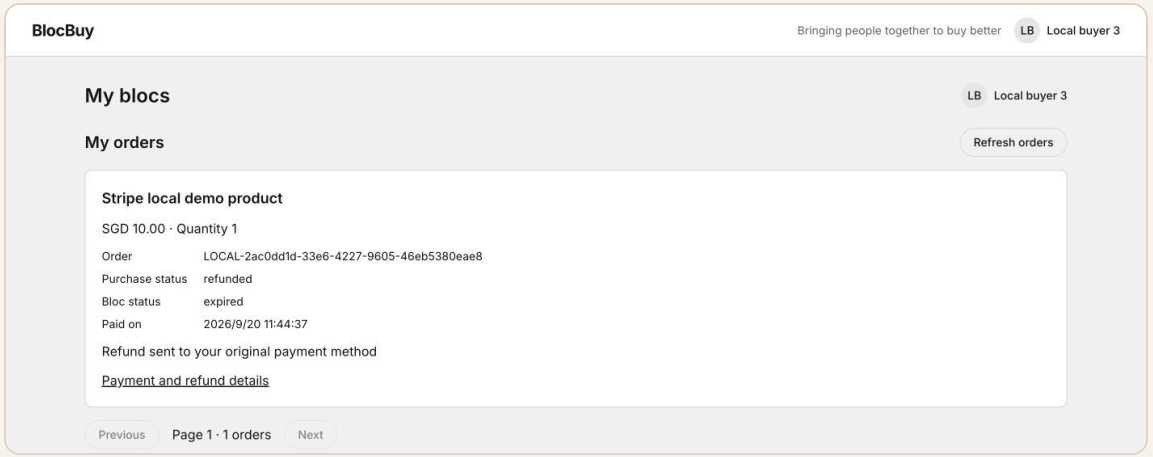
SWE5001 production topology



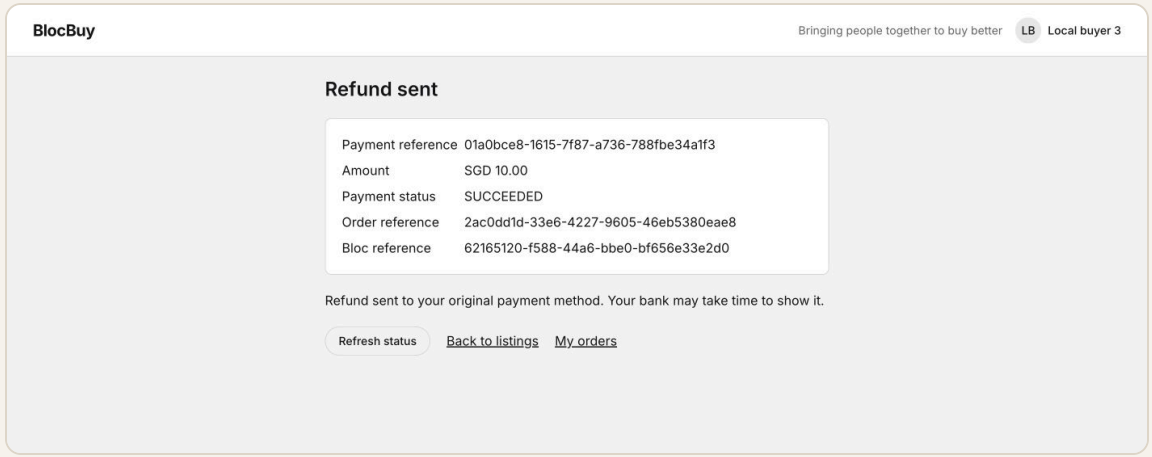
SWE5001 prod: the same code on ECS, rebuilt from scratch on every release and destroyed after it. SWE5006 dev is slide 29.

YQ Yorik

Stripe Sandbox refund, local evidence (20 Sep)



My orders: the expired bloc's order is refunded (UI of 20 Sep, since redesigned)



Receipt: "Refund sent", payment 01a0bce8..., SGD 10.00

Scenario	Result on 20 Sep (Team8 Stripe Sandbox, livemode=false)
START and JOIN	two SGD 10 payments → one member and one order each; the bloc reached succeeded, 2 of 2
Webhook replay	replayed valid events changed nothing; a modified payload with the old signature got 400
Expiry	an under-filled bloc expired → one SGD 10 refund obligation; its unpaid Checkout session expired
Refund	one full Sandbox refund `re_3UHbeV5iEVLN2uQJ1V71krwQ`, succeeded; a recheck created no duplicate

Local, not dev: real Stripe Sandbox calls from a local backend and PostgreSQL; no Google sign-in, no deployed environment. On dev the refund waits for #223 and #517. Source: `backend/docs/payment/stripe-sandbox-acceptance-20260920.md`

YQ Yorik

Scale-out and load results (SWE5001 ECS + local baseline)

SWE5001 PROD · ECS FARGATE

Scale-out

- Target tracking on ALB requests per target: 12,000 a minute, scale-out cooldown 60 s
- commerce 2–6 tasks in two AZs, notification 1–4 (read back 16 Oct)
- V3, 19 Oct: commerce **1 → 3 tasks** in about four minutes, no manual scaling; at 3 tasks ≈ 455 req/s, server p95 13–50 ms, 2 errors in 570,445
- Not V3 acceptance: the workstation generator dropped iterations, so the analyzer rejects the run
- V4 queue drain: `[FILL: backlog peak, notification tasks, drain time (19 Oct)]`

SWE5001 V2 · ECS, 16 OCT

Load, one task

- k6, ten one-minute plateaus from 10 to 200 requests a second, through CloudFront
- 57,600 requests, **0 errors**, run twice
- p95 between 23 and 57 ms on every plateau (target under 500 ms)
- No saturation reached: 200 a second is a tested lower bound, not a limit

DEV CODE · LOCAL RUN, 30 SEP

Workload baseline

100 users each	p95	Errors
Browse (list + detail)	24 ms	0
JOIN intent, unpaid	57 ms	0
My Orders	18 ms	0

30 paid teams expired within 58 s of their deadlines under that load. One laptop with real HTTP, PostgreSQL and Redis; **not a deployed result**.

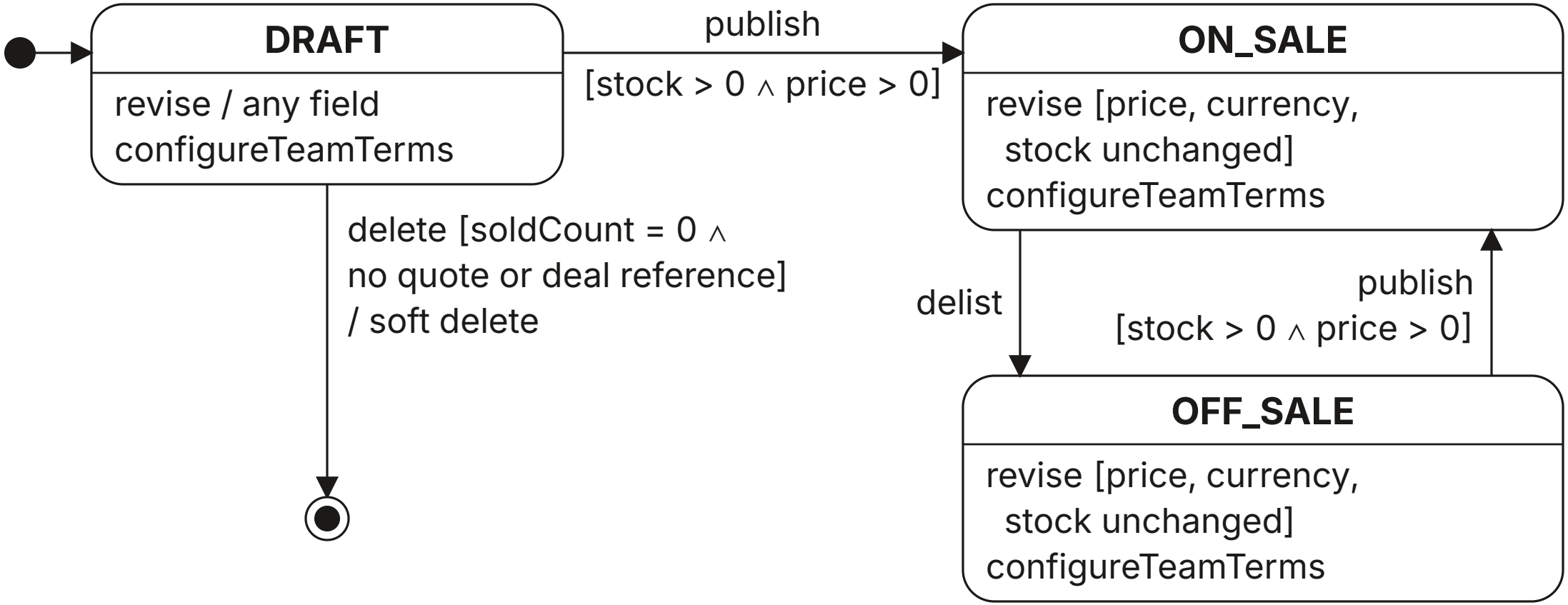
Dev, where the demo runs: one EC2 host by design (backend ADR 0008), so no auto-scaling there. Same code, two topologies: scale-out exists only in the SWE5001 topology and is not claimed for SWE5006 (slide 35).

Sources: docs/swe5001/evidence/20261016-http-baseline-dashboard/README.md (V2, scaling policy); docs/swe5001/evidence/20261019-v3-autoscaling/README.md (V3); docs/perf/sprint3-baseline.md (local workload).

LJ Liu Jin

UC9 listing statechart

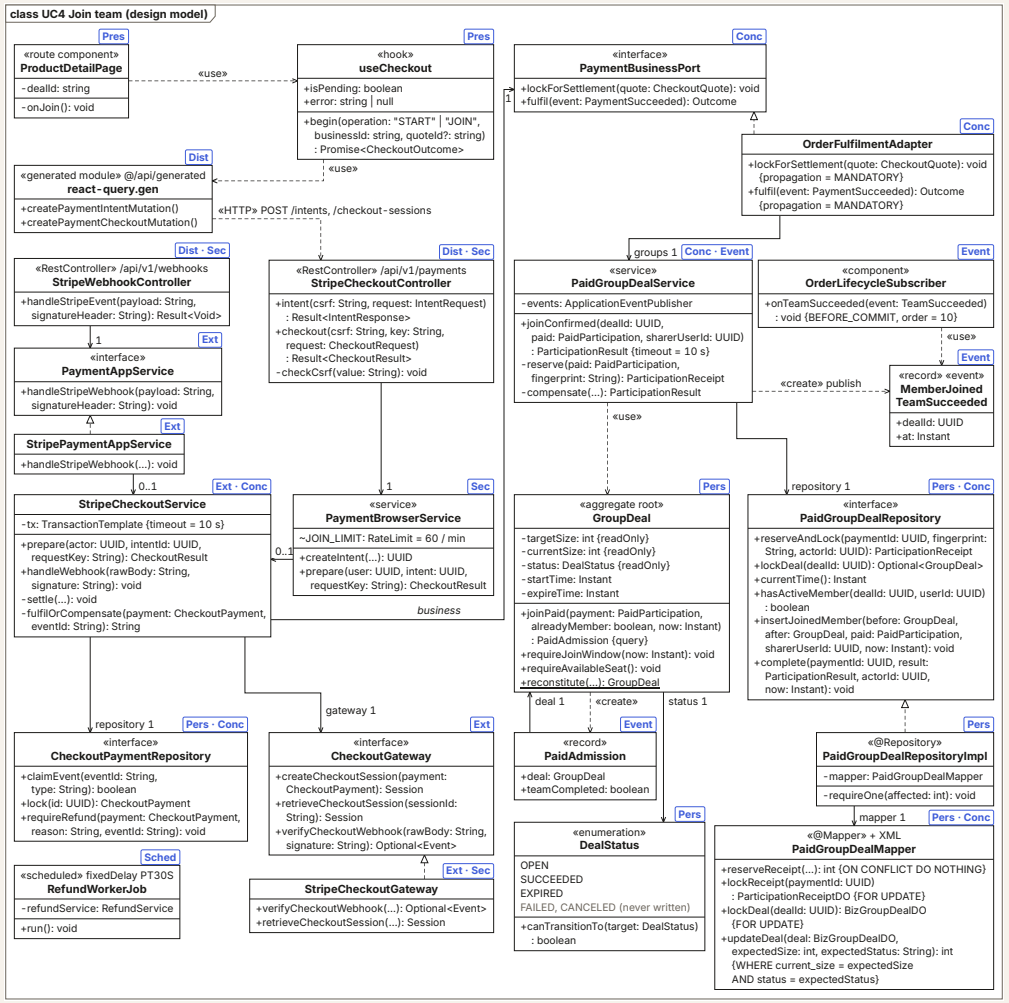
stm Listing lifecycle (Product.status)



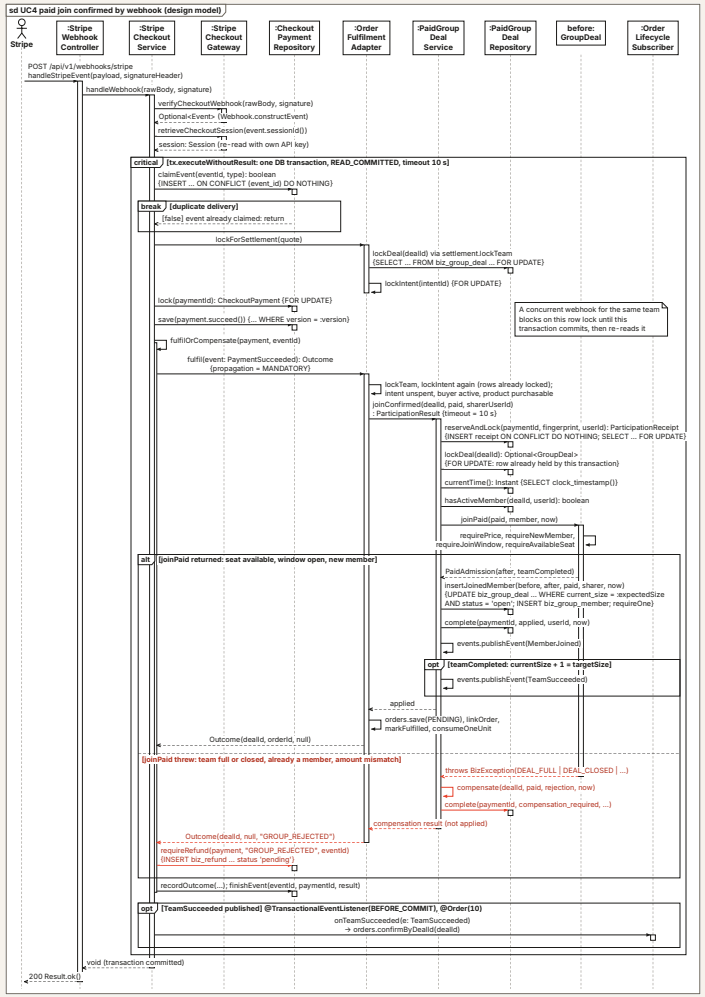
Nothing returns to DRAFT; any other event → 409, state unchanged.

WZ Eric

UC4 design class and sequence diagrams, complete



Design class diagram (2320 × 2314)



Design sequence (2320 × 3306)

Both are vector: open `src/diagrams/uc4-design-*.svg` in a browser tab and zoom. Every message carries its source line in the diagram spec. Report §3.4 (Eric).

WZ Eric

Every member’s use case and design problem

Member	Use case (report §3.4)	Design problem (report §3.5)	On a slide
Eric	UC3, UC4 start and join a team	Factory Method: team creation	UC4 models 13–16; pattern 17
Yorik	UC6 refund on team expiry	Observer: team outcome; refund idempotency as a mechanism (B6)	pattern 18; sequence B5
Pan	Receive notifications	Template Method: outbox transports	pattern 19
Lim	UC8 my orders and my teams (with UC7 frontend)	Strategy: UC1 pricing	pattern 20
Liu Jin	UC9 listing administration and team terms	State: listing lifecycle (full form)	patterns 21–22; statechart B13

Each member’s analysis class, analysis sequence, design class and design sequence diagrams are in their report section (template RT 3.4.1); the deck shows UC4 only (rubric G8). Owners: #463. `[VERIFY: UC9 owner: #32 names Eric, #463 names Liu Jin (open item 12)]`

LJ Liu Jin

Entity-relationship diagram, all 29 tables (scroll)

erd Full schema of the SWE5006 monolith (Flyway V1–V28, schema domain), by owning context

